A Project Report on

Spatio-Temporal Anomaly Detection In High Resolution Remote Sensing Imagery

Submitted in partial fulfillment for award of

Bachelor of Technology

Degree

in

CSE(Artificial Intelligence and Machine Learning)

By

J.Rakesh (Y22ACM434) **K.Shankar Ganesh (Y22ACM440)**

A.Srinadh (Y22ACM403) **G.Jyothika (Y22ACM433)**

Under the guidance of

Ms. G .V. Leela Kumari, M.Tech

Assistant Professor

Department of CSE(Artificial Intelligence and Machine Learning)

Bapatla Engineering College

(Autonomous)

(Affiliated to Acharya Nagarjuna University)

BAPATLA – 522 102, Andhra Pradesh, INDIA

2025-2026

Department of
CSE(Artificial Intelligence and Machine Learning)

CERTIFICATE

This is to certify that the project report entitled **Spatio-Temporal Anomaly Detection In High Resolution Remote Sensing Imagery** that is being submitted by J.Rakesh (Y22ACM434), K.Shankar Ganesh (Y22ACM440), A.Srinadh (Y22ACM403), G.Jyothika (Y22ACM433) in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science & Engineering to the Acharya Nagarjuna University is a record of bonafide work carried out by them under our guidance and supervision.

Date:

Signature of the Guide
Ms. G.V. Leela Kumari
Assistant Professor

Signature of the HOD
Prof. Dr. K. Mani Deep
Head Of The Department

DECLARATION

We declare that this project work is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

J.Rakesh (Y22ACM434)

K.Shankar Ganesh (Y22ACM440)

A.Srinadh (Y22ACM403)

G.Jyothika (Y22ACM433)

ACKNOWLEDGEMENT

We sincerely thank the following distinguished personalities who have given their advice and support for successful completion of the work.

We are deeply indebted to our most respected guide **Ms. G.V. Leela Kumari**, Assistant Professor, Department of CSE(AIML), for her valuable and inspiring guidance, comments, suggestions and encouragement.

We extend our sincere thanks to **Dr. K. Mani Deep**, Associate Professor, Head of the Dept. for extending his cooperation and providing the required resources.

We would like to thank our beloved Principal **Dr. B.Srinivasa Rao** for providing the online resources and other facilities to carry out this work.

We would like to sincerely thank our project coordinator **Ms. G.V.Leela Kumari**, Assistant Professor, Department of CSE(AIML), for her helpful suggestions in presenting this document.

We extend our sincere thanks to all other teaching faculty and non-teaching staff of the department, who helped directly or indirectly for their cooperation and encouragement.

J.Rakesh (Y22ACM434)

K.Shankar Ganesh (Y22ACM440)

A.Srinadh (Y22ACM403)

G.Jyothika (Y22ACM433)

ABSTRACT

Remote sensing using high-resolution satellite imagery enables critical applications such as disaster response, surveillance, and urban monitoring; however, the massive data volume makes manual analysis infeasible, necessitating automated anomaly detection. This project proposes STAD (Spatio-Temporal Anomaly Detection System), an object-centric framework that overcomes limitations of traditional pixel-based methods by integrating YOLO-OBB for oriented object detection, ByteTrack for temporal object association, CLIP for semantic feature embedding, and ChromaDB for persistent memory of normal object patterns. The system performs fully unsupervised anomaly detection by combining Kernel Density Estimation of spatial distributions with cosine-based semantic divergence, eliminating the need for labeled anomaly data. Evaluated on the NAIP 0.6m dataset across complex real-world scenarios including airports, ports, and naval bases, STAD achieves a mean Average Precision of 71.4%, an anomaly F1-score of 0.83, precision of 0.87, and a low false-positive rate of 9.2%, consistently outperforming existing baseline approaches.

TABLE OF CONTENTS

ABSTRACT v
LIST OF TABLES ix
LIST OF FIGURES x
CHAPTER – 1 INTRODUCTION 1
1.1 About the Project 1
1.2 Introduction to Remote Sensing and Anomaly Detection 1
1.3 Objectives of the Project 2
1.4 Problem Statement 3
1.5 Scope of the Project 4
CHAPTER – 2 LITERATURE REVIEW 5
2.1 Review of Previous Research 5
2.2 Pixel – Level Change Detection 5
2.3 Deep Reconstruction-Based Detection 6
2.4 CLIP – Based Zero – Shot Detection 7
2.5 Graph Neural Network Approaches 7
2.6 Comparative Analysis of Existing Approaches 8
2.7 Research Gap 9
CHAPTER - 3 SYSTEM ANALYSIS 11
3.1 Existing System 11
3.2 Proposed System 12
3.3 Feasibility Study 12
3.3.1 Technical Feasibility 12
3.3.2 Operational Feasibility 13
3.3.3 Economic Feasibility 13
3.4 Requirements 13
3.4.1 Hardware Requirements 13
3.4.2 Software Requirements 13
3.4.3 Functional Requirements 14
3.4.4 Non-Functional Requirements 15
CHAPTER – 4 SYSTEM DESIGN 16
4.1 High-Level System Architecture 16
4.2 Module Design 17
4.2.1 Scene Loader Module 17
4.2.2 Preprocessor Module 17

4.2.3 YOLO Detector Module 17
4.2.4 ByteTracker Module 17
4.2.5 CLIP Encoder Module 18
4.2.6 ChromaDB Module 18
4.2.7 Anomaly Scorer Module 18
4.3 Data Flow Diagram 18
4.4 UML Diagrams 19
4.4.1 Use Case Diagram 19
4.4.2 Sequence Diagram 20
4.4.3 Class Diagram 20
CHAPTER – 5 IMPLEMENTATION 22
5.1 Technologies Used 22
5.2 Modules Description 22
5.2.1 Scene Loader Module 22
5.2.2 Preprocessor Module 22
5.2.3 YOLO Detector Module 23
5.2.4 ByteTracker Module 23
5.2.5 CLIP Encoder Module 23
5.2.6 ChromaDB Module 23
5.2.7 Anomaly Scorer Module 23
5.3 Code Snippets 23
5.3.1 App.py 23
5.3.2 requirement.py 38
5.3.3 anomaly.py 39
5.3.4 detector.py 42
5.3.5 tracker.py 46
CHAPTER 6 – TESTING 50
6.1 Test Cases 50
6.2 Testing Methods 51
6.2.1 Unit Testing 51
6.2.2 Integration Testing 51
6.2.3 System Testing 51
6.3 Results and Analysis 51
CHAPTER 7 – RESULTS & DISCUSSION 53
7.1 Experimental Setup 53
7.2 Dataset Description 53
7.3 Object Detection Results 54

7.4 Anomaly Detection Results 54
7.5 Comparative Evaluation 55
7.6 Qualitative Results 56
CHAPTER 8 – CONCLUSION & FUTURE WORK 57
8.1 Conclusion 57
8.2 Future Work 58
8.2.1 Multispectral and SAR Imagery Integration 58
8.2.2 Graph Attention Networks for Relational Anomaly Modeling 58
8.2.3 Transformer-Based Spatial Attention 59
8.2.4 Real-Time Streaming Pipeline 59
8.2.5 Active Learning for Anomaly Label Collection 59
8.2.6 Web-Based Operational Interface 60
REFERENCES 61

LIST OF TABLES

Table 1 Comparative Analysis of Existing Anomaly Detection Approaches.....9
Table 2 Hardware Requirements For the STAD System.....13
Table 3 Software Requirements for the STAD System.....14
Table 4 Key Test Cases for the STAD System.....50
Table 5 NAIP 0.6m Evaluation Dataset Description.....53
Table 6 YOLO-OBB Object Detection Results per NAIP Scene.....54
Table 7 STAD Anomaly Detection Performance Metrics per Scene 55
Table 8 Comparative Evaluation of STAD Against Baseline Anomaly Detection Methods.....55

LIST OF FIGURES

Figure 1 High – Level System Architecture Diagram 16
Figure 2 Data Flow Diagram – Level 0 (Context Diagram) 19
Figure 3 UML Use Case Diagram 19
Figure 4 UML Sequence Diagram – Single Scene Anomaly Detection Run 20
Figure 5 UML Class Diagram – STAD Module Structure 21

CHAPTER – 1

INTRODUCTION

1.1 About the Project

The rapid advancement of satellite and aerial imaging technologies has resulted in an unprecedented volume of high-resolution remote sensing data being generated every day. This data serves as a critical resource for a wide range of applications including disaster response, military surveillance, urban development monitoring, environmental protection, and agricultural analysis. However, the sheer scale of this data makes human-driven interpretation impractical — analysts simply cannot be expected to manually review hundreds of satellite scenes daily without missing critical events or subtle changes that could have significant real-world consequences.

This project presents STAD — the Spatio-Temporal Anomaly Detection System — a fully automated, intelligent pipeline designed to detect unusual or unexpected occurrences in high-resolution satellite imagery without requiring any labeled anomaly data. The system is named to reflect its core strength: it reasons about both the spatial positioning of objects within a scene and their temporal behavior across multiple observations over time. Unlike traditional approaches that work at the pixel level, STAD reasons at the object level — identifying, tracking, and semantically understanding individual objects within a scene before determining whether their appearance or spatial location is anomalous.

The system was developed and evaluated using real-world NAIP 0.6m (National Agriculture Imagery Program) imagery sourced from Microsoft Planetary Computer, covering three geographically and operationally diverse scenes: San Diego International Airport, the Port of Long Beach container terminal, and San Diego Naval Base at 32nd Street. These scenes collectively span temporal imagery from 2012 to 2022, providing a rich multi-temporal dataset for evaluating the system's ability to detect object-level anomalies across changing real-world environments.

1.2 Introduction to Remote Sensing and Anomaly Detection

Remote sensing refers to the acquisition of information about the Earth's surface from a distance, typically through satellite or airborne sensors. Modern remote sensing

platforms — such as those operated by the United States Geological Survey (USGS) and the National Agriculture Imagery Program — are capable of capturing sub-meter-resolution optical imagery across vast geographic areas, making them invaluable for monitoring dynamic environments, tracking land use changes, and detecting events of strategic or environmental significance.

Anomaly detection in remote sensing is the process of identifying objects, events, or patterns in satellite imagery that deviate significantly from what is considered normal for a given scene or region. Such anomalies can take many forms: the unauthorized appearance of military vehicles in a civilian port, the sudden construction of new structures in a protected area, the clustering of an unusually large number of aircraft on an airport apron, or the appearance of unfamiliar vessel types in a container terminal. Early and accurate detection of these events is critical for timely intervention, security monitoring, and environmental protection.

Traditional anomaly detection methods in remote sensing fall into three broad categories. Pixel-level statistical methods compare multi-temporal images by thresholding spectral differences, but are easily disrupted by lighting changes, atmospheric conditions, and sensor noise. Deep reconstruction models such as Convolutional LSTM autoencoders learn normal appearance patterns and flag high reconstruction errors as anomalies, but struggle to provide semantic context and often misclassify rare yet normal textures. Supervised change detection networks require expensive labeled before-and-after image pairs, limiting their applicability in new or rapidly evolving domains. STAD is designed to overcome all of these limitations.

1.3 Objectives of the Project

The following objectives guided the design, development, and evaluation of the STAD system:

- To develop an end-to-end object-centric anomaly detection pipeline that reasons at the level of individual objects rather than raw pixels or image patches.
- To integrate YOLO-OBB oriented object detection for accurately localizing objects at arbitrary rotation angles in aerial imagery, overcoming the limitations of axis-aligned bounding box detectors.

- To apply ByteTrack multi-object tracking to establish persistent object identities across temporal satellite image snapshots, enabling temporal change reasoning at the object level.
- To encode each detected object using the CLIP vision-language model, producing 512-dimensional semantic embeddings that capture both visual appearance and contextual meaning for downstream anomaly scoring.
- To maintain a persistent vector database of normal object embeddings using ChromaDB, enabling efficient cosine-similarity-based retrieval for historical context comparison without requiring labeled anomaly examples.
- To develop a composite anomaly scoring function that combines spatial kernel density estimation with per-object contextual divergence, producing interpretable object-attributed anomaly heatmaps.
- To evaluate the proposed framework on real-world NAIP 0.6m scenes from three operationally significant locations and compare its performance against established baseline methods.

1.4 Problem Statement

High-resolution satellite imagery is continuously generated across nearly every region of the globe, yet the automated detection of meaningful object-level anomalies within this data remains an unsolved challenge. Existing detection systems operate at the pixel or patch level, making them incapable of answering object-level questions such as: "Has a type of vessel that has never appeared in this harbor suddenly emerged?" or "Is this aircraft parked in a location where aircraft have never historically been seen?".

Pixel-based methods are highly sensitive to environmental noise such as seasonal lighting variations, cloud cover, and atmospheric distortion, leading to excessive false positives that overwhelm analysts. Reconstruction-based deep learning models lack semantic awareness and cannot describe what triggered an anomaly alert. Supervised approaches depend on annotated datasets that are expensive to create and rarely available for new geographic regions or event types.

Furthermore, no existing system simultaneously addresses all four critical requirements of object-level detection, temporal tracking, semantic understanding, and unsupervised anomaly scoring within a single end-to-end pipeline. There is a pressing need for a framework that can identify anomalous objects in satellite imagery

accurately, provide interpretable explanations of why each object was flagged, and operate without requiring labeled anomaly examples — precisely the gaps that STAD is designed to fill.

1.5 Scope of the Project

The scope of this project encompasses the complete design, implementation, and evaluation of the STAD pipeline. The system is designed to operate in an unsupervised manner, requiring only a reference set of normal scene observations without any labeled anomaly data. The scope includes:

- Fine-tuning of YOLO-OBB on real-world NAIP 0.6m imagery for oriented object detection across three operationally significant scenes.
- Implementation of ByteTrack for multi-object tracking across temporal image snapshots spanning 2012 to 2022.
- Extraction of CLIP-based semantic embeddings for each detected object and storage in a persistent ChromaDB vector database.
- Development and evaluation of the composite KDE-based spatial and semantic anomaly scoring mechanism.
- Evaluation of the complete pipeline on three NAIP scenes: San Diego International Airport (Scene ID: 678822c6ae), Port of Long Beach (Scene ID: 9e752f465e), and San Diego Naval Base (Scene ID: 26ed3e829a).

The current scope focuses on optical satellite imagery at 0.6m native resolution. Future extensions of this work may include integration of multispectral or SAR (Synthetic Aperture Radar) imagery, deployment of Graph Attention Networks for relational anomaly modeling, and development of a real-time operational monitoring interface.

CHAPTER – 2

LITERATURE REVIEW

2.1 Review of Previous Research

Over the past decade, the problem of detecting anomalies in satellite and aerial imagery has attracted significant research attention from the remote sensing, computer vision, and artificial intelligence communities. The earliest work in this area was motivated by practical surveillance and environmental monitoring needs — researchers wanted automated tools that could flag unusual events in satellite imagery without requiring a human analyst to manually review every scene. What began as relatively simple statistical comparisons has evolved, through waves of innovation, into sophisticated deep learning pipelines capable of reasoning about complex multi-temporal scenes.

The progression of research in this field broadly follows the evolution of machine learning itself. Early pixel-level statistical methods gave way to deep reconstruction models as neural networks demonstrated superior pattern learning capabilities. More recently, the emergence of foundation models such as CLIP has opened new directions for zero-shot and unsupervised anomaly detection. In parallel, advances in multi-object tracking and scalable vector database technologies have expanded the toolkit available to researchers working on spatio-temporal analysis of satellite imagery. The STAD system proposed in this project draws inspiration from each of these directions and synthesizes them into a novel unified pipeline.

2.2 Pixel – Level Change Detection

Pixel-level change detection is one of the oldest and most widely used techniques in remote sensing analysis. The core idea is conceptually simple: compare two images of the same area taken at different times and highlight regions where significant spectral differences are observed. Methods such as image differencing, ratioing, and spectral index comparison — including the widely used Normalized Difference Vegetation Index (NDVI) — have been applied extensively for monitoring agricultural land, forest cover, and urban growth over multi-year periods.

Change Vector Analysis (CVA) extended this idea by treating each pixel as a multi-dimensional feature vector, enabling the direction of change — not just its magnitude — to be used as a discriminating signal. These approaches are computationally inexpensive, easy to interpret, and produce visually clear change maps that are accessible to non-technical stakeholders. However, their fundamental weakness is the complete absence of semantic understanding. A change in pixel intensity could be caused by a genuine anomaly — a new structure being built, a ship entering a harbor — but it could equally well be caused by a shift in solar illumination angle, a passing cloud shadow, seasonal vegetation changes, or minor sensor calibration differences between acquisition dates. This inability to distinguish meaningful change from environmental noise is the root cause of the persistently high false-positive rates observed in all pixel-level approaches.

2.3 Deep Reconstruction-Based Detection

As deep learning gained momentum in the mid-2010s, researchers began exploring reconstruction-based approaches for anomaly detection in satellite imagery. The central insight behind these methods is that a model trained exclusively on normal images should be able to reconstruct normal inputs with high fidelity, but will produce high reconstruction error when presented with anomalous inputs that fall outside the training distribution. This reconstruction error is then used as an anomaly signal.

Convolutional autoencoders applied to satellite image patches demonstrated promising results for detecting structural damage after natural disasters such as earthquakes and floods, where the visual difference between pre-event and post-event scenes is dramatic. ConvLSTM-based autoencoders took this approach further by incorporating temporal sequences, learning motion and appearance patterns over time and flagging deviations as anomalies. These models represented a genuine advance over pixel-level methods in terms of their ability to handle complex visual patterns and reduce sensitivity to minor illumination variations.

Despite these advances, reconstruction-based methods carry two persistent limitations. First, they tend to flag rare but entirely normal textures as anomalies simply because the model was not exposed to enough examples of those textures during training. A seldom-photographed rural airstrip, for instance, might trigger a false alarm simply

because the model has rarely encountered it. Second — and more critically — reconstruction error is a fundamentally low-level signal. It can tell an analyst that something looks unusual in a patch of pixels, but it cannot say what that something is, what kind of object is involved, or why the situation is anomalous. This absence of semantic interpretability is a major practical barrier to deployment.

2.4 CLIP – Based Zero – Shot Detection

The introduction of CLIP (Contrastive Language-Image Pretraining) by Radford et al. in 2021 marked a significant turning point in computer vision. By training on hundreds of millions of image-text pairs collected from the internet, CLIP developed a rich shared embedding space in which semantically similar images and text descriptions are mapped to nearby locations, regardless of the specific visual domain from which they originated. This generalization capability makes CLIP exceptionally powerful for tasks where labeled training data is scarce or entirely unavailable.

WinCLIP was among the first systems to operationalize CLIP for anomaly detection, applying it to industrial defect detection by comparing patch-level embeddings against text descriptions of normal and anomalous appearances. The results demonstrated that CLIP could identify manufacturing defects without any exposure to labeled defect examples during training. The remote sensing community quickly recognized the potential of similar approaches for satellite image classification, scene understanding, and change detection.

However, applying CLIP directly to spatio-temporal anomaly detection in satellite imagery presents significant challenges that are not present in the single-image industrial inspection domain. CLIP generates embeddings for individual image patches or crops, but it has no built-in mechanism for tracking objects over time or comparing new observations against a historical baseline of normal appearances. Without these capabilities, CLIP can flag an individual object crop as semantically unusual, but it cannot determine whether that object has been seen before in the same location, whether its current position is spatially inconsistent with historical patterns, or whether its appearance has gradually changed across multiple observations. These temporal and historical reasoning gaps are precisely what the STAD system is designed to address.

2.5 Graph Neural Network Approaches

Graph Neural Networks (GNNs) have emerged as a powerful paradigm for modeling relational information in complex scenes. In the context of remote sensing, GNNs have been applied to model spatial relationships between objects or regions within a satellite image, representing each detected region as a node in a graph and encoding spatial proximity or semantic similarity as weighted edges. This allows the model to reason about whether a particular object makes sense given its neighbors — a type of contextual reasoning that single-object classifiers are inherently unable to perform.

GNN-based anomaly detection methods have shown strong results on structured datasets where scene layouts are relatively consistent and predictable, such as organized port facilities or military installations with well-defined spatial patterns. By incorporating relational context into the anomaly score, these methods go beyond simple per-object appearance analysis and consider the broader scene composition — an approach that can be particularly effective when anomalies are defined by the unusual combination of objects rather than the unusual appearance of any individual object.

However, GNN-based approaches carry significant practical drawbacks that have limited their widespread adoption. They require carefully predefined graph structures that must be designed for each specific scene type, a labor-intensive process that does not generalize well across diverse geographic regions or operational domains. They also tend to be computationally expensive when applied to the large, high-resolution images produced by modern satellite platforms. The STAD system achieves contextual and relational reasoning through dynamic vector retrieval in ChromaDB, which provides similar benefits with considerably less computational overhead and without requiring any manually designed graph topology.

2.6 Comparative Analysis of Existing Approaches

Table 2.6 below summarizes the key characteristics of each reviewed approach category in terms of their ability to handle the core challenges of satellite anomaly detection. This comparison makes clear that while individual approaches address some aspects of the problem, none of them offers a complete solution that simultaneously

combines object-level detection, temporal tracking, semantic understanding, and unsupervised anomaly scoring within a single end-to-end pipeline.

Table 1 Comparative Analysis of Existing Anomaly Detection Approaches

Approach	Core Technique	Strengths	Weaknesses
Pixel-Level Methods	Image differencing, spectral thresholding	Simple, fast, interpretable	No semantic understanding, high false-positive rate
Convolutional Autoencoder	Patch reconstruction error	Learns normal patterns without labels	No semantic *Table 2 Comparative Analysis of Existing Anomaly Detection Approaches* context, flags rare-but-normal textures
ConvLSTM Autoencoder	Temporal sequence reconstruction	Captures temporal dynamics	High false positives, no object-level reasoning
CLIP-Only	Vision-language zero-shot embedding	Rich semantic embeddings, zero-shot capable	No temporal tracking, no historical context comparison
GNN-Based Detection	Graph-based relational modeling	Models relational scene context	Needs predefined graphs, computationally expensive
STAD (Proposed)	Object detection + tracking + CLIP + ChromaDB + KDE	Object-level, semantic, temporal, unsupervised, interpretable	Sensitive to severe occlusion; requires reference scenes

2.7 Research Gap

After a thorough review of the existing literature, a clear and consequential research gap becomes evident. While each of the component technologies — oriented object detection, multi-object tracking, vision-language embeddings, and vector database retrieval — is individually well-developed and has demonstrated strong results in its respective domain, no prior work has attempted to integrate all four of these capabilities into a single unified pipeline specifically designed for satellite image anomaly detection.

Existing systems either work at the pixel level and lack semantic awareness, or they work with semantic embeddings but lack the temporal tracking and historical context comparison mechanisms needed to identify object-level anomalies reliably. None of the reviewed approaches produce object-attributed anomaly heatmaps that

allow an analyst to immediately understand not only where an anomaly occurred but which specific object caused it and why it was flagged. This interpretability gap is as significant as the accuracy gap for real-world operational deployment.

The STAD system directly addresses this research gap by presenting the first complete pipeline that integrates YOLO-OBB oriented detection, ByteTrack persistent tracking, CLIP semantic embedding, and ChromaDB vector retrieval into a unified framework with a composite anomaly scoring function validated on real-world NAIP 0.6m satellite imagery.

CHAPTER - 3

SYSTEM ANALYSIS

3.1 Existing System

Before introducing the proposed STAD system, it is important to examine the landscape of methods that have been commonly used for anomaly detection in satellite imagery. A thorough understanding of how these systems work — and more critically, where they fail — provides the necessary context for appreciating why the STAD approach is both necessary and effective.

Existing methods can be organized into four broad categories. The first is pixel-level statistical detection, the oldest and most widely deployed category. These methods work by directly comparing images taken at different times and measuring how much individual pixels have changed, using techniques such as image differencing, spectral thresholding, and change vector analysis. They are simple, computationally inexpensive, and produce visually interpretable change maps. However, they are entirely blind to the objects present in a scene and are easily confused by routine environmental variations.

The second category is threshold and rule-based anomaly detection, where predefined statistical baselines are used to flag unusual patterns. Traffic monitoring systems or port surveillance tools might raise an alert when the number of objects in a region exceeds a historical average. These methods work reasonably well in highly stable environments with predictable object distributions, but they become unreliable in dynamic or geographically diverse scenes where the definition of "normal" varies significantly over time and across locations.

The third category — deep reconstruction-based detection — uses neural network architectures, primarily convolutional autoencoders and ConvLSTM models, to learn the appearance of normal scenes and flag regions with high reconstruction error as anomalous. These methods are more capable than simple pixel comparisons, but they still operate on image patches rather than individual objects, and they cannot provide semantic descriptions of detected anomalies.

The fourth category is supervised change detection, where models are trained on labeled pairs of before-and-after images. These systems tend to achieve strong performance in the specific domain they were trained on, but they generalize poorly to new geographic regions or event types, and their reliance on expensive annotated training data limits their practical applicability.

3.2 Proposed System

The STAD system is designed with a clear philosophy: approach satellite imagery the way a trained human analyst would — not as a grid of pixels to be statistically compared, but as a collection of meaningful objects that can be individually identified, tracked across time, and evaluated against a learned understanding of what is normal for that scene and location.

Rather than asking "which pixels have changed?", STAD asks "which objects are behaving unusually, and in what way?" The pipeline integrates four core technologies: YOLO-OBB for oriented object detection, ByteTrack for persistent multi-object tracking, CLIP for semantic embedding of detected objects, and ChromaDB for historical context retrieval. These are combined with a composite anomaly scoring function that produces interpretable, object-attributed anomaly heatmaps.

Critically, the system operates in a fully unsupervised manner. No labeled anomaly examples are required at any stage of the pipeline. The system learns what is normal from the reference population of observed scenes and flags any object whose spatial position or semantic appearance diverges significantly from that learned baseline.

3.3 Feasibility Study

3.3.1 Technical Feasibility

All component technologies used in STAD — YOLOv8-OBB, ByteTrack, CLIP ViT-B/32, and ChromaDB — are mature, open-source frameworks with active developer communities. The NAIP 0.6m dataset is publicly accessible through Microsoft Planetary Computer. The required hardware (NVIDIA GPU, 16+ GB RAM) is readily available in academic settings, confirming strong technical feasibility.

3.3.2 Operational Feasibility

The system's unsupervised design means it can be deployed in any new geographic region without annotation overhead. The modular pipeline architecture allows individual components to be updated or replaced without disrupting the rest of the system. The qualitative output (color-coded anomaly heatmaps) is directly usable by domain analysts without requiring deep technical expertise.

3.3.3 Economic Feasibility

All software components are open-source with no licensing costs. NAIP imagery is freely accessible through Microsoft Planetary Computer. The primary hardware cost is a CUDA-capable GPU workstation, which is standard equipment in academic and research institutions.

3.4 Requirements

3.4.1 Hardware Requirements

Table 2 Hardware Requirements For the STAD System

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i5 10th Gen / AMD Ryzen 5	Intel Core i7 10th Gen+ / AMD Ryzen 7
GPU	NVIDIA GTX 1080 (8 GB VRAM)	NVIDIA RTX 3060 / RTX 3080 (8–10 GB VRAM)
RAM	16 GB DDR4	32 GB DDR4
Storage	50 GB SSD	256 GB NVMe SSD
Network	10 Mbps broadband	100 Mbps broadband
OS	Ubuntu 20.04 LTS / Windows 10 (WSL2)	Ubuntu 22.04 LTS

3.4.2 Software Requirements

Table 3 Software Requirements for the STAD System

Software / Library	Version	Purpose
Python	3.10+	Primary programming language
PyTorch + CUDA	2.0 + CUDA 11.8	Deep learning framework for YOLO-OBB and CLIP
Ultralytics YOLOv8-OBB	8.0+	Oriented bounding box object detection
OpenAI CLIP (ViT-B/32)	Latest	512-dim semantic object embedding
ByteTrack	Latest	Multi-object tracking across temporal frames
ChromaDB	0.4+	Persistent vector database for embedding storage
Rasterio	1.3+	GeoTIFF satellite image reading and CRS handling
NumPy / SciPy	Latest	KDE computation, normalization, distance metrics

3.4.3 Functional Requirements

- FR-1: The system must load NAIP 0.6m GeoTIFF imagery from Microsoft Planetary Computer using scene identifiers.
- FR-2: The system must normalize pixel values, tile large scenes into overlapping 1024×1024 sub-images, and apply coordinate reference system transformations.
- FR-3: The system must run YOLO-OBB inference and return oriented bounding box detections described by (xc, yc, w, h, θ), class label, and confidence score.

- FR-4: The system must run ByteTrack to assign and maintain persistent track identifiers across temporal frames.
- FR-5: The system must extract oriented crops for each tracked object, encode them through CLIP ViT-B/32, and produce L2-normalized 512-dimensional embedding vectors.
- FR-6: The system must store reference scene embeddings in ChromaDB and retrieve the 10 most similar reference embeddings by cosine similarity.
- FR-7: The system must compute per-object composite anomaly scores using $A_i = 0.6 \times s_dens(i) + 0.4 \times d_ctx(i)$ and classify objects with $A_i > 0.65$ as anomalous.
- FR-8: The system must produce output visualizations with color-coded oriented bounding boxes (green/yellow/red) and a continuous anomaly heatmap.
- FR-9: The system must compute and report mAP@0.50, F1-score, precision, recall, and false-positive rate for the evaluation scenes.

3.4.4 Non-Functional Requirements

- NFR-1 Performance: System must complete inference, embedding, and scoring for a single 1024×1024 tile in under 3 seconds on recommended hardware.
- NFR-2 Accuracy: Anomaly detection module must achieve F1-score ≥ 0.80 and FPR < 15% on evaluation scenes. Detection module must achieve mAP@0.50 ≥ 65%.
- NFR-3 Scalability: System must process scenes from new geographic locations without pipeline modification.
- NFR-4 Reliability: System must handle edge cases gracefully, including empty scenes, cloud-occluded frames, and sparse ChromaDB collections.
- NFR-5 Reproducibility: All random seeds must be fixed and documented. Model checkpoints and ChromaDB collections must be serializable for independent verification.

CHAPTER – 4

SYSTEM DESIGN

4.1 High-Level System Architecture

The STAD system is organized into three conceptual layers — a Data Layer, a Processing Layer, and an Output Layer — that together define the complete information flow from raw satellite imagery to actionable anomaly visualization. Figure 6.1 presents the high-level architecture of the system, showing how data moves through each layer and how the individual processing modules interact.

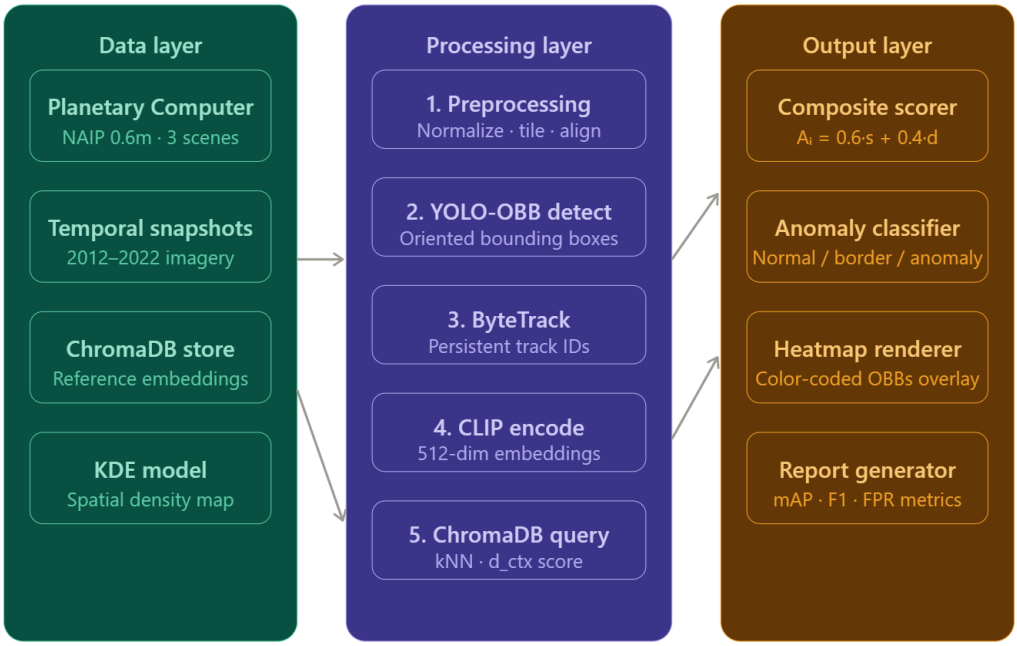

Figure 1 High – Level System Architecture Diagram

The Data Layer is responsible for all interactions with external data sources, including the Microsoft Planetary Computer NAIP imagery API and the local ChromaDB persistent embedding store. The Processing Layer contains the five core computation modules: preprocessing and tiling, YOLO-OBB detection, ByteTrack tracking, CLIP encoding, and ChromaDB retrieval. The Output Layer synthesizes the spatial density scores from the KDE model and the contextual divergence scores from ChromaDB into the final composite anomaly assessment and visualization.

4.2 Module Design

Each module in the processing layer is designed as an independent component with a well-defined interface, enabling modular testing, replacement, and future extension. The following subsections describe the design of each module in detail.

4.2.1 Scene Loader Module

The Scene Loader module handles all interactions with the Microsoft Planetary Computer API. It accepts a scene ID and temporal range specification as input, authenticates with the Planetary Computer endpoint using the planetary-computer Python SDK, and downloads the corresponding NAIP 0.6m GeoTIFF files for all requested temporal snapshots. The module is also responsible for spatial alignment of multi-temporal snapshots using Rasterio's coordinate reference system transformation utilities, ensuring that all temporal snapshots of a given scene are perfectly registered to a common spatial coordinate system before processing.

4.2.2 Preprocessor Module

The Preprocessor module accepts a loaded and aligned GeoTIFF scene and converts it into a sequence of overlapping 1024×1024 pixel tiles suitable for GPU inference. Tiles are generated with a 128-pixel overlap to avoid boundary artifacts in the detection output. Pixel intensity values are normalized to the [0, 1] range, and tile coordinate offsets are recorded so that detection results can be mapped back to the original scene coordinate system after inference.

4.2.3 YOLO Detector Module

The YOLO Detector module wraps the Ultralytics YOLOv8-OBB inference interface. It accepts a preprocessed image tile as input and returns a list of oriented bounding box detections, each described by center coordinates, dimensions, rotation angle, class label, and confidence score. Detections below the confidence threshold of 0.25 are filtered before the results are returned. The module also handles the coordinate remapping required to convert tile-relative detection coordinates back to the full-scene coordinate system using the tile offset metadata produced by the Preprocessor.

4.2.4 ByteTracker Module

The ByteTracker module maintains a persistent tracking state across sequential calls corresponding to different temporal snapshots of the same scene. On each call, it receives the full set of detections from the current snapshot, applies the ByteTrack two-

stage association algorithm, and returns a list of tracked objects with their assigned track IDs and updated Kalman filter state estimates. Track histories are maintained in memory across all temporal snapshots for a given scene evaluation run.

4.2.5 CLIP Encoder Module

The CLIP Encoder module accepts an oriented bounding box crop as input, applies perspective correction to produce a rectangular crop aligned with the object's principal axis, resizes it to 224×224 pixels, and encodes it using the CLIP ViT-B/32 image encoder to produce a 512-dimensional embedding vector. The embedding is L2-normalized before being returned for downstream use.

4.2.6 ChromaDB Module

The ChromaDB module manages two distinct collections: a reference collection populated with embeddings from normal scenes, and an evaluation collection used during anomaly scoring. The store() method adds new embeddings to the reference collection with associated metadata including scene ID, track ID, and temporal snapshot index. The query() method retrieves the k most similar reference embeddings for a given evaluation embedding using ChromaDB's built-in cosine similarity index and returns both the embeddings and their similarity scores for divergence computation.

4.2.7 Anomaly Scorer Module

The Anomaly Scorer module receives the complete set of tracked and embedded objects for a scene, fits a 2D Gaussian KDE over the spatial centroids of all reference scene objects, and computes per-object spatial density scores and contextual divergence scores. It then applies the composite scoring formula to produce a final anomaly score for each object and classifies objects as normal, borderline, or anomalous based on the $\delta = 0.65$ threshold. It returns a structured results object containing all scores and classifications, which is passed to the visualization module.

4.3 Data Flow Diagram

Figure 4.3 shows the Level-0 context diagram for the STAD system, illustrating the system's external interactions at the highest level of abstraction. The system receives satellite scene identifiers and configuration parameters from the analyst, downloads the corresponding NAIP imagery from the Planetary Computer, and produces anomaly heatmaps and detection reports as output.

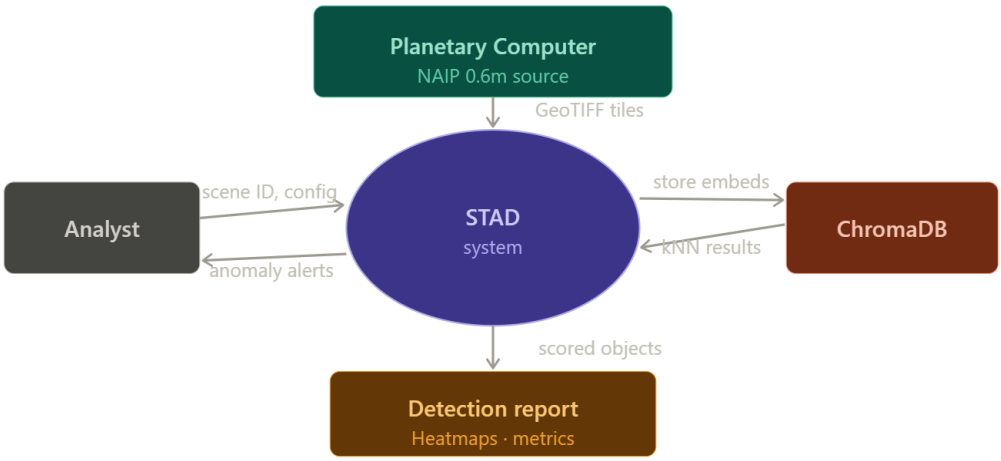

Figure 2 Data Flow Diagram – Level 0 (Context Diagram)

4.4 UML Diagrams

4.4.1 Use Case Diagram

Figure 4.4.1 presents the UML use case diagram for the STAD system, identifying the two primary actors — the Analyst who initiates and configures system runs, and the System itself which executes the automated detection pipeline — and the use cases associated with each.

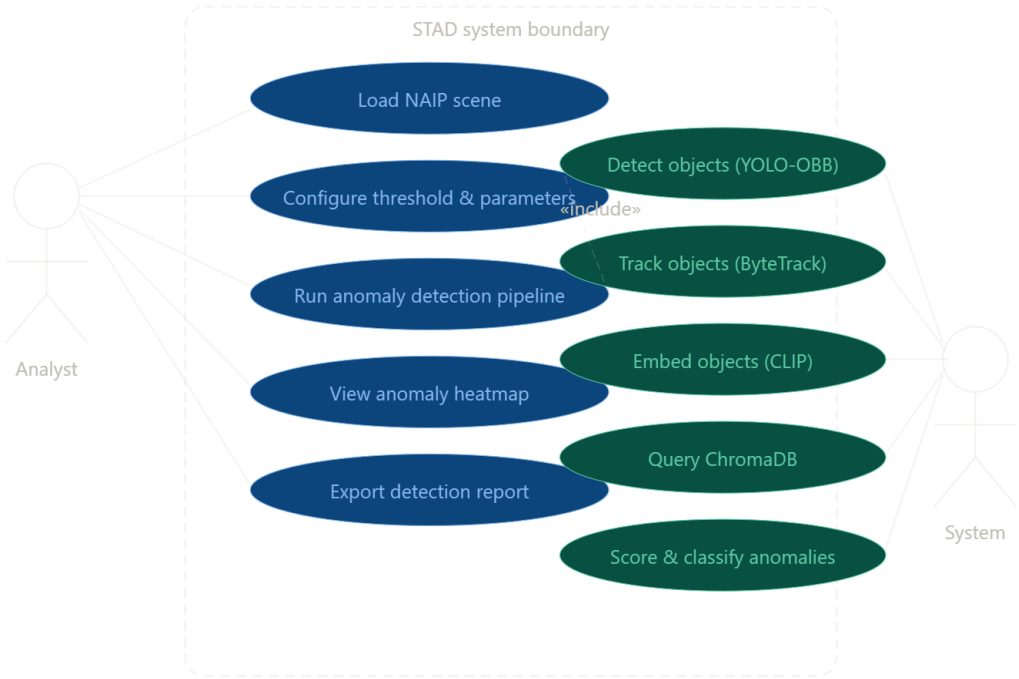

Figure 3 UML Use Case Diagram

4.4.2 Sequence Diagram

Figure 4.4.2 shows the UML sequence diagram for a single scene anomaly detection run, tracing the message flow between the seven principal components of the STAD pipeline from scene loading through to heatmap generation.

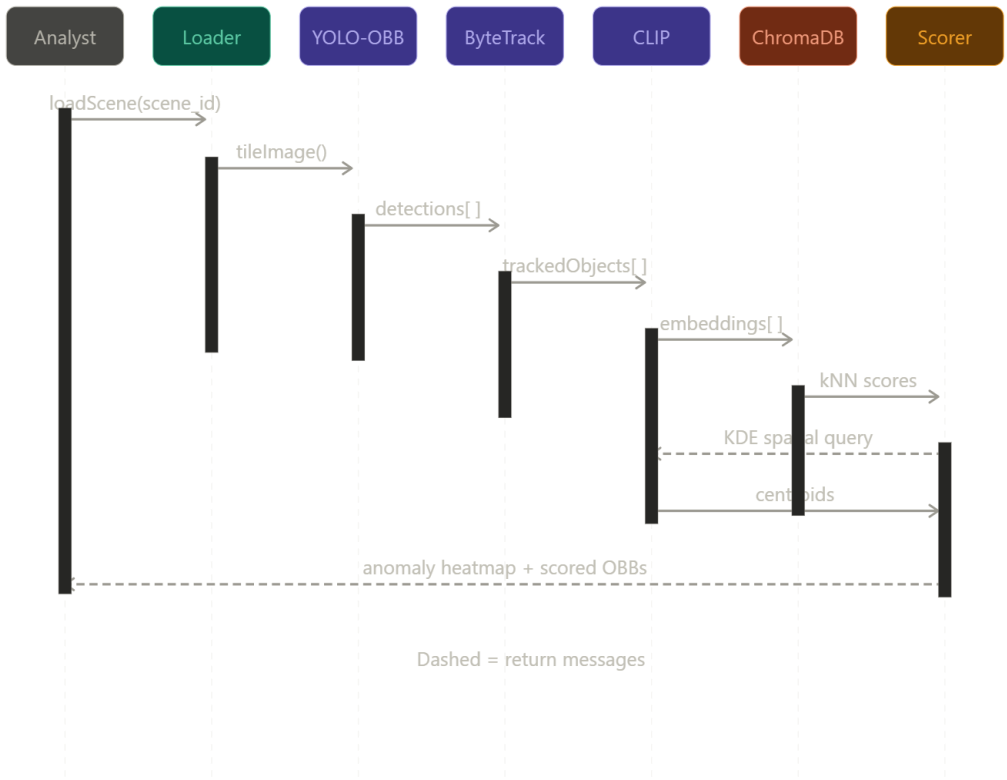

Figure 4 UML Sequence Diagram – Single Scene Anomaly Detection Run

4.4.3 Class Diagram

Figure 4.4.3 presents the UML class diagram, showing the principal classes in the STAD codebase, their attributes, methods, and inter-class relationships. Each processing module is implemented as an independent Python class, maintaining a clean separation of concerns across the pipeline.

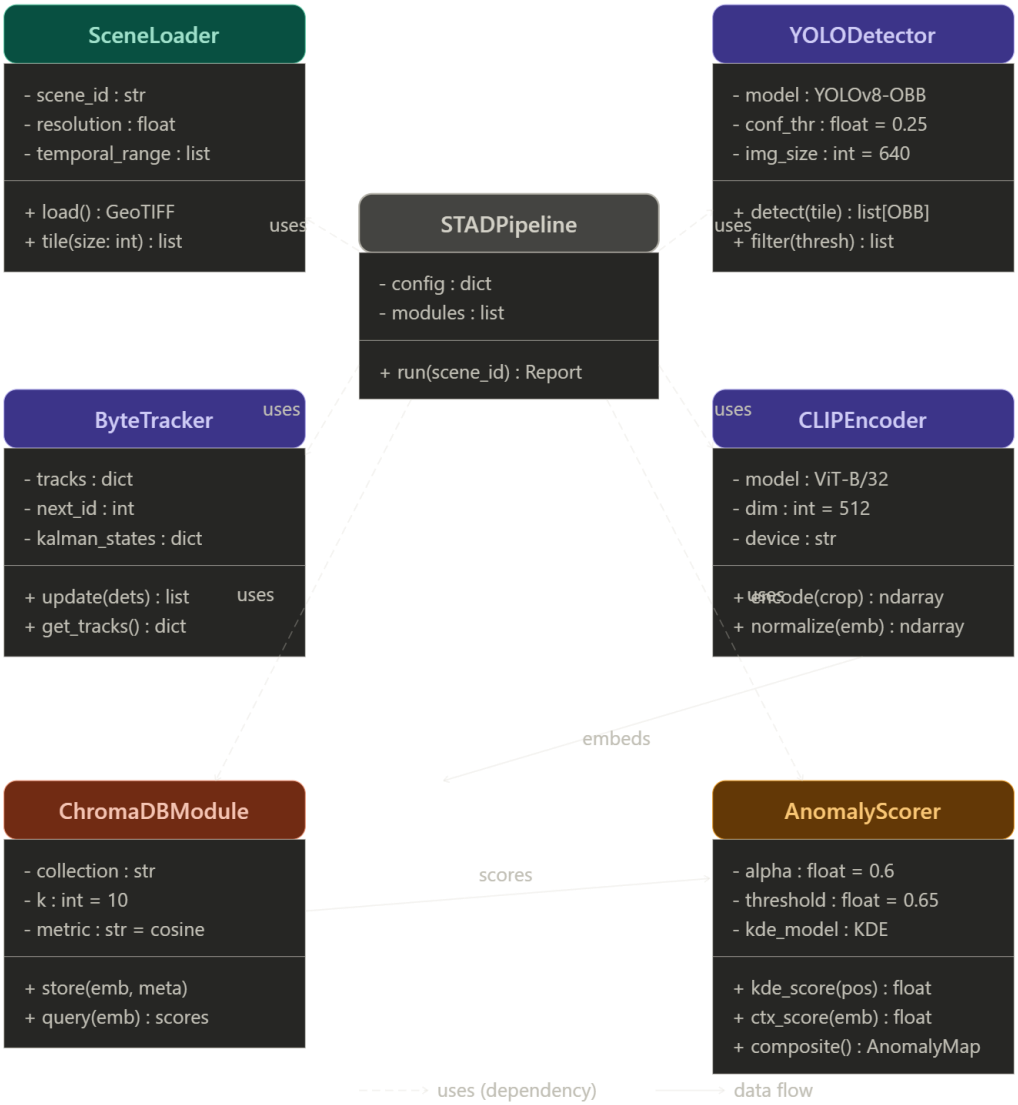

Figure 5 UML Class Diagram – STAD Module Structure

CHAPTER – 5

IMPLEMENTATION

5.1 Technologies Used

The STAD pipeline was implemented in Python 3.10 using PyTorch 2.0 with CUDA 11.8 acceleration. The complete implementation is organized into a Python package with seven modules corresponding to the processing components described in Chapter 4. A main pipeline orchestrator script manages the sequential execution of modules, accepting a YAML configuration file as input that specifies the scene IDs, temporal ranges, confidence thresholds, KDE bandwidth, and anomaly scoring parameters.

Key technology choices include: YOLOv8-OBB via Ultralytics for oriented object detection; ByteTrack for persistent multi-object tracking; OpenAI CLIP ViT-B/32 in inference-only mode without fine-tuning to preserve broad semantic generalization; ChromaDB with cosine distance metric and persistent filesystem backend; Rasterio for GeoTIFF handling; SciPy for Gaussian KDE computation; and OpenCV for visualization.

5.2 Modules Description

5.2.1 Scene Loader Module

The Scene Loader module handles all interactions with the Microsoft Planetary Computer API. It accepts a scene ID and temporal range specification as input, authenticates with the Planetary Computer endpoint, and downloads the corresponding NAIP 0.6m GeoTIFF files for all requested temporal snapshots. The module is also responsible for spatial alignment of multi-temporal snapshots using Rasterio's coordinate reference system transformation utilities.

5.2.2 Preprocessor Module

The Preprocessor module converts a loaded GeoTIFF scene into a sequence of overlapping 1024×1024 pixel tiles suitable for GPU inference. Tiles are generated with a 128-pixel overlap to avoid boundary artifacts. Pixel intensity values are normalized to the [0, 1] range, and tile coordinate offsets are recorded so that detection results can be mapped back to the original scene coordinate system after inference.

5.2.3 YOLO Detector Module

The YOLO Detector module wraps the Ultralytics YOLOv8-OBB inference interface. It accepts a preprocessed image tile and returns a list of oriented bounding box detections. Detections below the confidence threshold of 0.25 are filtered. The module handles coordinate remapping to convert tile-relative detection coordinates back to the full-scene coordinate system using tile offset metadata.

5.2.4 ByteTracker Module

The ByteTracker module maintains a persistent tracking state across sequential calls corresponding to different temporal snapshots of the same scene. On each call, it receives detections from the current snapshot, applies the ByteTrack two-stage association algorithm, and returns a list of tracked objects with assigned track IDs and updated Kalman filter state estimates.

5.2.5 CLIP Encoder Module

The CLIP Encoder module accepts an oriented bounding box crop, applies perspective correction to produce a rectangular crop aligned with the object's principal axis, resizes it to 224×224 pixels, and encodes it using CLIP ViT-B/32 to produce a 512-dimensional embedding vector. The embedding is L2-normalized before being returned.

5.2.6 ChromaDB Module

The ChromaDB module manages two distinct collections: a reference collection populated with embeddings from normal scenes, and an evaluation collection used during anomaly scoring. The store() method adds new embeddings with metadata. The query() method retrieves the $k = 10$ most similar reference embeddings using ChromaDB's cosine similarity index.

5.2.7 Anomaly Scorer Module

The Anomaly Scorer module receives the complete set of tracked and embedded objects, fits a 2D Gaussian KDE over the spatial centroids of all reference scene objects, and computes per-object spatial density scores and contextual divergence scores. It applies the composite scoring formula and classifies objects as normal, borderline, or anomalous based on the $\delta = 0.65$ threshold.

5.3 Code Snippets

5.3.1 App.py

```
import streamlit as st
```

```
import numpy as np
import cv2
import json
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from PIL import Image
from datetime import datetime

from utils.config import get_config
from utils.preprocessing import enhance_optical
from utils.visualization import (
    draw_detections, draw_trajectories, draw_anomaly_heatmap,
    create_dashboard, RISK_COLORS,
)
from utils.real_data import (
    pixel_to_latlon, set_active_region,
    load_highres_scenes,
)
from utils.scene_manager import load_cached_scenes
from utils.map_overlay import create_threat_map
from utils.change_detection import compute_change_maps,
compute_temporal_stats
from models.detector import YOLOv8OBBDetector
from models.tracker import ByteTracker
from models.transformer import TemporalAnalyzer
from models.anomaly import RiskScorer, SpatialAnomalyMapper
try:
    from streamlit_folium import st_folium
    FOLIUM_AVAILABLE = True
except ImportError:
    FOLIUM_AVAILABLE = False

st.set_page_config(
    page_title="STAD - Temporal Anomaly Detection",
    page_icon="\U0001f6f0\ufe0f",
    layout="wide",
    initial_sidebar_state="expanded",
)
st.markdown("""
```

```
<style>
    @import
url(\'https://fonts.googleapis.com/css2?family=JetBrains+Mono:wght@40
0;700&family=Rajdhani:wght@400;600;700&display=swap\' );
    .main .block-container { padding-top: 0.8rem; max-width: 100%; }
    .cmd-header { background: linear-gradient(135deg, #040810 0%,
#0c1624 40%, #0a1018 100%); border: 1px solid #1a3050; border-left:
4px solid #00d4ff; border-radius: 6px; padding: 18px 28px; margin-
bottom: 16px; }
    .cmd-header h1 { font-family: \'Rajdhani\', sans-serif; color:
#00d4ff; font-size: 1.9em; margin: 0; letter-spacing: 4px; text-
transform: uppercase; }
    .cmd-header .sub { font-family: \'JetBrains Mono\', monospace;
color: #4a7090; font-size: 0.78em; margin: 4px 0 0 0; }
    .kpi-card { background: linear-gradient(145deg, #0c1420,
#10202e); border: 1px solid #1a3050; border-radius: 6px; padding:
14px 16px; text-align: center; }
    .kpi-card .val { font-family: \'JetBrains Mono\', monospace;
font-size: 1.8em; font-weight: 700; color: #00d4ff; }
    .kpi-card .lbl { font-family: \'Rajdhani\', sans-serif; font-
size: 0.78em; color: #4a7090; text-transform: uppercase; letter-
spacing: 1px; }
    .risk-critical { color: #ff3333; font-weight: bold; }
    .risk-high     { color: #ff8800; font-weight: bold; }
    .risk-medium   { color: #ffcc00; font-weight: bold; }
    .risk-low      { color: #33cc33; }
    .status-live { display: inline-block; width: 8px; height: 8px;
background: #00ff88; border-radius: 50%; animation: blink 1.5s
infinite; margin-right: 6px; }
    @keyframes blink { 0%, 100% { box-shadow: 0 0 0 0
rgba(0,255,136,0.4); } 50% { box-shadow: 0 0 0 6px rgba(0,255,136,0);
} }
    section[data-testid="stSidebar"] { background: linear-
gradient(180deg, #060c14 0%, #0c1824 100%); }
    section[data-testid="stSidebar"] .stMarkdown h1, section[data-
testid="stSidebar"] .stMarkdown h2, section[data-testid="stSidebar"]
.stMarkdown h3 { color: #00d4ff; font-family: \'Rajdhani\', sans-
serif; }
    div[data-testid="stExpander"] { border: 1px solid #1a3050;
border-radius: 6px; }
</style>
""", unsafe_allow_html=True)
REGIONS = {
    "San Diego Intl Airport — 190 planes detected (0.6m NAIP)": {
        "min_lon": -117.205, "max_lon": -117.180, "min_lat": 32.728,
"max_lat": 32.740,
        "center_lat": 32.734, "center_lon": -117.1925, "name": "San
Diego Intl Airport — 190 planes detected (0.6m NAIP)",
```

```
    },
    "Port of Long Beach — 64 ships, 96 vehicles (0.6m NAIP)": {
        "min_lon": -118.230, "max_lon": -118.205, "min_lat": 33.742,
"max_lat": 33.758,
        "center_lat": 33.750, "center_lon": -118.2175, "name": "Port
of Long Beach — 64 ships, 96 vehicles (0.6m NAIP)",
    },
    "San Diego Naval Base — USS carriers & destroyers (0.6m NAIP)": {
        "min_lon": -117.128, "max_lon": -117.112, "min_lat": 32.683,
"max_lat": 32.696,
        "center_lat": 32.6895, "center_lon": -117.120, "name": "San
Diego Naval Base — USS carriers & destroyers (0.6m NAIP)",
    },
}
with st.sidebar:
    st.markdown("## Configuration")
    st.markdown("---")


    region_choice = st.selectbox("Region", list(REGIONS.keys()),
index=0)
    selected_region = {"name": region_choice,
**REGIONS[region_choice]}
    st.info(
        f"**{selected_region[\"name\"]}**\n\n"
        f"Lat: {selected_region[\"min_lat\"]:.2f} -
{selected_region[\"max_lat\"]:.2f} N\n\n"
        f"Lon: {selected_region[\"min_lon\"]:.2f} -
{selected_region[\"max_lon\"]:.2f} E"
    )
    st.markdown("---")
    num_frames = st.slider("Temporal Frames", 5, 20, 16, 1)
    num_objects = st.slider("Target Count", 4, 20, 12)
    with st.expander("Detection Settings"):
        conf_threshold = st.slider("Confidence", 0.05, 0.9, 0.15,
0.05)
        iou_threshold = st.slider("IoU", 0.1, 0.9, 0.45, 0.05)
    with st.expander("Anomaly Thresholds"):
        risk_low = st.slider("Low Cutoff", 0.1, 0.5, 0.3, 0.05)
        risk_med = st.slider("Medium Cutoff", 0.3, 0.8, 0.6, 0.05)
        risk_high = st.slider("High Cutoff", 0.5, 0.95, 0.85, 0.05)
    st.markdown("---")
    run_pipeline = st.button("RUN TEMPORAL ANOMALY DETECTION",
use_container_width=True, type="primary")
def cv2_to_pil(img):
```

```
    if img is None: return Image.new("RGB", (256, 256), (20, 20, 30))
    if len(img.shape) == 2: return Image.fromarray(img)
    return Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))


def risk_hex(level):
    return {"LOW": "#33cc33", "MEDIUM": "#ffcc00", "HIGH": "#ff8800",
"CRITICAL": "#ff3333"}.get(level, "#888")


def _plotly_dark(fig, height=350):
    fig.update_layout(
        height=height, plot_bgcolor="#060c14",
paper_bgcolor="#060c14",
        font=dict(color="#8aa0b8", family="JetBrains Mono,
monospace", size=11),
        margin=dict(l=40, r=20, t=30, b=40),
    )
    return fig
if not run_pipeline:
    st.markdown("### How Temporal Anomaly Detection Works")
    st.markdown("""
This system analyses **multi-temporal aerial imagery** to detect
anomalous
behaviour patterns over time. Each frame is a real NAIP 0.6m aerial
acquisition from a different year.
)
    st.markdown("---")
    st.markdown("### Input Data Preview")
    preview_scenes = load_highres_scenes(region=selected_region,
max_scenes=8)
    if preview_scenes:
        st.success(f"Loaded {len(preview_scenes)} temporal scenes for
**{selected_region[\"name\"]}** "
                   f"({preview_scenes[0][\"date\"]} to
{preview_scenes[-1][\"date\"]})")
        st.markdown("**NAIP 0.6m Aerial — Multi-Year
Acquisitions:**")
        cols = st.columns(min(len(preview_scenes), 8))
        for i, scene in enumerate(preview_scenes[:8]):
            with cols[i]:
                st.caption(scene["date"])
                opt = scene["optical"]
                st.image(cv2.cvtColor(opt, cv2.COLOR_BGR2RGB) if
len(opt.shape) == 3 else opt, use_container_width=True)
```

```
    else:
        st.warning("No high-res data for this region. Select a region
with downloaded data.")
    st.info("Click **RUN TEMPORAL ANOMALY DETECTION** in the sidebar
to start the pipeline.")
else:
    config = get_config()
    config.num_demo_frames = num_frames
    config.image_size = (2048, 2048)
    config.detector.confidence_threshold = conf_threshold
    config.detector.iou_threshold = iou_threshold
    config.detector.input_size = (640, 640)
    config.anomaly.risk_threshold_low = risk_low
    config.anomaly.risk_threshold_medium = risk_med
    config.anomaly.risk_threshold_high = risk_high
    config.anomaly.risk_threshold_critical = 0.95
    config.anomaly.risk_weights = {"speed": 0.2, "direction": 0.2,
"loitering": 0.2, "convergence": 0.2, "transformer": 0.2}
    config.tracker.max_age = 30; config.tracker.min_hits = 3


    set_active_region(selected_region)
    detector = YOLOv8OBBDetector(config.detector); tracker =
ByteTracker(config.tracker)
    temporal_analyzer = TemporalAnalyzer(config.transformer);
risk_scorer = RiskScorer(config.anomaly)
    spatial_anomaly_mapper = SpatialAnomalyMapper(config.anomaly)


    status_text = st.empty(); status_text.info("Loading temporal
scenes...")
    temporal_scenes = load_cached_scenes(selected_region,
config.num_demo_frames)
    if not temporal_scenes: st.error("Could not load temporal
scenes."); st.stop()
    actual_frames = len(temporal_scenes)
    st.success(f"Loaded {actual_frames} temporal scenes for
**{selected_region[\"name\"]}** "
               f"({temporal_scenes[0][\"date\"]} to
{temporal_scenes[-1][\"date\"]})")


    all_frame_detections = []; all_frame_tracks = [];
all_frame_enhanced_optical = []; all_frame_change_maps = []
    progress_bar = st.progress(0); st.markdown("### Processing
Frames")
    for i, scene in enumerate(temporal_scenes):
```

```
        status_text.text(f"Processing frame {i+1}/{actual_frames}
({scene[\"date\"]})...")

        optical_img = scene["optical"]

        enhanced_optical_img = enhance_optical(optical_img);
all_frame_enhanced_optical.append(enhanced_optical_img)

        detections = detector.detect(enhanced_optical_img);
all_frame_detections.append(detections)

        tracks = tracker.update(detections);
all_frame_tracks.append(tracks)

        if i > 0:

            prev_optical = temporal_scenes[i-1]["optical"]

            change_map = compute_change_maps(prev_optical,
optical_img)

            all_frame_change_maps.append(change_map)

        else:
all_frame_change_maps.append(np.zeros(optical_img.shape[:2],
dtype=np.float32))

        progress_bar.progress((i + 1) / actual_frames)

    status_text.success("Detection, Tracking, and Change Detection
complete.")


    temporal_stats = compute_temporal_stats(temporal_scenes)

    status_text.info("Running Temporal Transformer Analysis...")

    trajectories =
temporal_analyzer.process_trajectories(all_frame_tracks,
temporal_stats)

    status_text.success("Temporal Transformer Analysis complete.")


    status_text.info("Calculating Anomaly Scores...")

    risk_results = risk_scorer.score_anomalies(trajectories,
temporal_stats)

    status_text.success("Anomaly Scoring complete.")


    status_text.info("Generating Spatial Anomaly Heatmap...")

    heatmap, hotspots =
spatial_anomaly_mapper.generate_heatmap(risk_results, trajectories,
config.image_size)

    status_text.success("Spatial Anomaly Heatmap generated.")

    # Visualization

    status_text.info("Generating Visualizations...")

    last_frame_idx = actual_frames - 1

    last_optical = temporal_scenes[last_frame_idx]["optical"]

    last_detections = all_frame_detections[last_frame_idx]


    det_vis = draw_detections(last_optical.copy(), last_detections)
```

```
    traj_vis = draw_trajectories(last_optical.copy(), trajectories,
risk_results)
    heatmap_vis = draw_anomaly_heatmap(last_optical.copy(), heatmap)


    total_detections = sum(len(d) for d in all_frame_detections)
    total_tracks = len(trajectories)
    risk_counts = {"LOW": 0, "MEDIUM": 0, "HIGH": 0, "CRITICAL": 0}
    for res in risk_results.values(): risk_counts[res["risk_level"]]
+= 1
    detection_counts = [len(d) for d in all_frame_detections]
    track_counts = [len(t) for t in all_frame_tracks]


    st.markdown("### Key Performance Indicators")
    kpi1, kpi2, kpi3, kpi4 = st.columns(4)
    with kpi1: st.markdown(f"<div class=\"kpi-card\"><div
class=\"val\">{total_detections}</div><div class=\"lbl\">Total
Detections</div></div>", unsafe_allow_html=True)
    with kpi2: st.markdown(f"<div class=\"kpi-card\"><div
class=\"val\">{total_tracks}</div><div class=\"lbl\">Total
Tracks</div></div>", unsafe_allow_html=True)
    with kpi3: st.markdown(f"<div class=\"kpi-card\"><div
class=\"val\">{risk_counts[\"CRITICAL\"]}</div><div
class=\"lbl\">Critical Anomalies</div></div>",
unsafe_allow_html=True)
    with kpi4: st.markdown(f"<div class=\"kpi-card\"><div
class=\"val\">{risk_counts[\"HIGH\"]}</div><div class=\"lbl\">High
Anomalies</div></div>", unsafe_allow_html=True)


    st.markdown("---")
    st.markdown("### Latest Frame Visualizations")
    col1, col2 = st.columns(2)
    with col1: st.markdown("**Detections (YOLOv8-OBB)**");
st.image(cv2_to_pil(det_vis), use_container_width=True)
    with col2:
        st.markdown("**Zoomed Detections**")
        zoom_cols = st.columns(min(len(last_detections), 4))
        n_det = len(last_detections)
        last_det_vis = draw_detections(last_optical.copy(),
last_detections, draw_labels=False)
        for di in range(n_det):
            det = last_detections[di]; col = zoom_cols[di %
min(n_det, 4)]
            h_img, w_img = last_det_vis.shape[:2]; crop_r = 128
            x1 = max(0, int(det.cx) - crop_r); y1 = max(0,
int(det.cy) - crop_r)
```

```
            x2 = min(w_img, int(det.cx) + crop_r); y2 = min(h_img,
int(det.cy) + crop_r)

            crop = last_det_vis[y1:y2, x1:x2]

            if crop.size > 0: col.image(cv2_to_pil(cv2.resize(crop,
(512, 512), interpolation=cv2.INTER_CUBIC)),
use_container_width=True, caption=f"{det.class_name}
({det.confidence:.2f})")


    st.markdown("---"); st.markdown("### Temporal Anomaly Detection
Results")

    c1, c2 = st.columns(2)

    with c1: st.markdown("**Trajectory Trails (Risk-Colored)**");
st.image(cv2_to_pil(traj_vis), use_container_width=True)

    with c2: st.markdown("**Spatial Anomaly Heatmap**");
st.image(cv2_to_pil(heatmap_vis), use_container_width=True)


    st.markdown("#### Anomaly Scores per Tracked Object")

    threat_rows = []

    for tid, data in sorted(risk_results.items(), key=lambda x:
x[1]["risk_score"], reverse=True):

        bd = data["anomaly_breakdown"]

        threat_rows.append({

            "Track": tid, "Class": data["class_name"], "Risk Score":
f"{data[\"risk_score\"]:.4f}",

            "Level": data["risk_level"], "Speed":
f"{bd[\"speed\"]:.3f}", "Direction": f"{bd[\"direction\"]:.3f}",

            "Loitering": f"{bd[\"loitering\"]:.3f}", "Convergence":
f"{bd[\"convergence\"]:.3f}", "Transformer":
f"{bd[\"transformer\"]:.3f}",

        })

    if threat_rows: st.dataframe(pd.DataFrame(threat_rows),
use_container_width=True, hide_index=True)


    st.markdown("---"); st.markdown("### Temporal Analytics")

    c1, c2, c3 = st.columns(3)

    with c1:

        st.markdown("**Risk Distribution**")

        fig_risk = px.bar(x=list(risk_counts.keys()),
y=list(risk_counts.values()), color=list(risk_counts.keys()),

                          color_discrete_map={"LOW": "#33cc33",
"MEDIUM": "#ffcc00", "HIGH": "#ff8800", "CRITICAL": "#ff3333"})

        fig_risk = _plotly_dark(fig_risk, 300);
fig_risk.update_layout(showlegend=False, xaxis_title="Risk Level",
yaxis_title="Count")

        st.plotly_chart(fig_risk, use_container_width=True)

    with c2:
```

```
        st.markdown("**Detections & Tracks Over Time**")

        fig_det = go.Figure()

        fig_det.add_trace(go.Scatter(x=list(range(1,
actual_frames+1)), y=detection_counts, mode="lines+markers",
name="Detections", line=dict(color="#00d4ff", width=2)))

        fig_det.add_trace(go.Scatter(x=list(range(1,
actual_frames+1)), y=track_counts, mode="lines+markers",
name="Tracks", line=dict(color="#00ff88", width=2)))

        fig_det = _plotly_dark(fig_det, 300);
fig_det.update_layout(xaxis_title="Frame", yaxis_title="Count")

        st.plotly_chart(fig_det, use_container_width=True)

    with c3:

        st.markdown("**Anomaly Radar — Top 3 Threats**")

        top3 = sorted(risk_results.items(), key=lambda x:
x[1]["risk_score"], reverse=True)[:3]

        fig_radar = go.Figure(); cats = ["Speed", "Direction",
"Loitering", "Convergence", "Transformer"]

        for tid, data in top3:

            bd = data["anomaly_breakdown"]

            vals = [bd["speed"], bd["direction"], bd["loitering"],
bd["convergence"], bd["transformer"]]; vals.append(vals[0])

            fig_radar.add_trace(go.Scatterpolar(r=vals, theta=cats +
[cats[0]], fill="toself", name=f"T{tid} [{data[\"risk_level\"]}]",
opacity=0.6))

        fig_radar = _plotly_dark(fig_radar, 300);
fig_radar.update_layout(polar=dict(bgcolor="#060c14",
radialaxis=dict(visible=True, range=[0, 1], color="#3a5a7a"),
angularaxis=dict(color="#8aa0b8")))

        st.plotly_chart(fig_radar, use_container_width=True)


    st.markdown("---"); st.markdown("### Detection Class Matrix &
Confidence Analysis")

    from models.detector import DOTA_CLASSES

    detected_classes = sorted({d.class_name for frame_dets in
all_frame_detections for d in frame_dets})

    if detected_classes:

        matrix_data = [[sum(1 for d in frame_dets if d.class_name ==
cls) for frame_dets in all_frame_detections] for cls in
detected_classes]

        cm1, cm2 = st.columns(2)

        with cm1:

            st.markdown("**Class × Frame Detection Matrix**")

            frame_labels = [f"F{i+1}" for i in
range(len(all_frame_detections))]

            fig_matrix = go.Figure(data=go.Heatmap(z=matrix_data,
x=frame_labels, y=detected_classes, colorscale="YlOrRd",
```

```
text=matrix_data, texttemplate="%{text}", textfont={"size": 11,
"color": "white"}, hovertemplate="Class: %{y}

Frame: %{x}

Count: %{z}<extra></extra>"))

            fig_matrix = _plotly_dark(fig_matrix, max(300,
len(detected_classes) * 30 + 100));
fig_matrix.update_layout(xaxis_title="Temporal Frame",
yaxis_title="Object Class")

            st.plotly_chart(fig_matrix, use_container_width=True)

        with cm2:

            st.markdown("**Per-Class Confidence Distribution**")

            conf_data = [{"Class": d.class_name, "Confidence":
d.confidence} for frame_dets in all_frame_detections for d in
frame_dets]

            if conf_data:

                df_conf = pd.DataFrame(conf_data)

                fig_conf = px.box(df_conf, x="Class", y="Confidence",
color="Class", points="outliers")

                fig_conf = _plotly_dark(fig_conf, max(300,
len(detected_classes) * 30 + 100));
fig_conf.update_layout(showlegend=False, xaxis_tickangle=-45,
xaxis_title="Object Class", yaxis_title="Confidence")

                st.plotly_chart(fig_conf, use_container_width=True)


        st.markdown("**Inter-Class Spatial Co-occurrence Matrix**")

        st.caption("Normalized co-occurrence: how often two classes
appear in the same spatial region (128px proximity)")

        n_cls = len(detected_classes); cls_idx = {c: i for i, c in
enumerate(detected_classes)}

        cooccur = np.zeros((n_cls, n_cls), dtype=np.float64)

        for frame_dets in all_frame_detections:

            for i, d1 in enumerate(frame_dets):

                for d2 in frame_dets[i+1:]:

                    if np.sqrt((d1.cx - d2.cx)**2 + (d1.cy -
d2.cy)**2) < 128:

                        ci, cj = cls_idx[d1.class_name],
cls_idx[d2.class_name]; cooccur[ci][cj] += 1; cooccur[cj][ci] += 1

        row_sums = cooccur.sum(axis=1, keepdims=True);
row_sums[row_sums == 0] = 1; cooccur_norm = cooccur / row_sums

        fig_cooccur =
go.Figure(data=go.Heatmap(z=np.round(cooccur_norm, 2).tolist(),
x=detected_classes, y=detected_classes, colorscale="Viridis",
text=np.round(cooccur_norm, 2).tolist(), texttemplate="%{text}",
textfont={"size": 10, "color": "white"}, hovertemplate="%{y} ↔ %{x}:
%{z:.2f}<extra></extra>"))

        fig_cooccur = _plotly_dark(fig_cooccur, max(400, n_cls * 35 +
100)); fig_cooccur.update_layout(xaxis_title="Class",
yaxis_title="Class", xaxis_tickangle=-45)
```

```
        st.plotly_chart(fig_cooccur, use_container_width=True)


    st.markdown("---"); st.markdown("### Object Trajectories")
    c1, c2 = st.columns([3, 2])
    with c1:
        fig_traj = go.Figure()
        for tid, tdata in trajectories.items():
            traj = tdata["trajectory"]; xs = [pt["cx"] for pt in
traj]; ys = [pt["cy"] for pt in traj]
            rl = risk_results.get(tid, {}).get("risk_level", "LOW")
            fig_traj.add_trace(go.Scatter(x=xs, y=ys,
mode="lines+markers", name=f"T{tid} ({tdata[\"class_name\"]})
[{rl}]", line=dict(color=risk_hex(rl), width=2),
marker=dict(size=3)))
        fig_traj = _plotly_dark(fig_traj, 500);
fig_traj.update_layout(yaxis=dict(autorange="reversed"),
xaxis_title="X (pixels)", yaxis_title="Y (pixels)", title="All
Tracked Object Paths")
        st.plotly_chart(fig_traj, use_container_width=True)
    with c2:
        if hotspots:
            st.markdown("**Anomaly Hotspots**")
            st.dataframe(pd.DataFrame([{"Rank": i+1, "Location":
f"({hs[\"center_x\"]:.0f}, {hs[\"center_y\"]:.0f})", "Intensity":
f"{hs[\"intensity\"]:.4f}"} for i, hs in enumerate(hotspots[:10])]),
use_container_width=True, hide_index=True)
        st.markdown("**Heatmap (Interactive)**")
        fig_heat = px.imshow(heatmap,
color_continuous_scale="YlOrRd", labels=dict(color="Anomaly"),
aspect="equal")
        fig_heat = _plotly_dark(fig_heat, 300);
st.plotly_chart(fig_heat, use_container_width=True)


    st.markdown("---"); st.markdown("### Geo-Intelligence Map")
    try: st_folium(create_threat_map(risk_results, trajectories,
geo_info, hotspots), width=None, height=500, returned_objects=[])
    except Exception as e: st.warning(f"Map rendering failed: {e}")


    st.markdown("---"); st.markdown("### Export Report")
    report = {
        "system": "STAD - Spatio-Temporal Anomaly Detection",
"timestamp": datetime.now().isoformat(), "region":
selected_region["name"],
        "pipeline": {"frames_analyzed": actual_frames, "date_range":
f"{temporal_scenes[0][\"date\"]} to {temporal_scenes[-1][\"date\"]}",
"total_days": temporal_stats.get("total_days", 0)},
```

```
        "results": {"total_tracks": len(risk_results),
"risk_distribution": risk_counts, "hotspots_detected":
len(hotspots)},

        "threats": [{

            "track_id": tid, "class": data["class_name"],
"risk_score": round(data["risk_score"], 4), "risk_level":
data["risk_level"],

            "anomaly_breakdown": {k: round(v, 4) for k, v in
data["anomaly_breakdown"].items()},

        } for tid, data in sorted(risk_results.items(), key=lambda x:
x[1]["risk_score"], reverse=True)],

    }


    def generate_pdf_report(report_data, det_vis_img, heatmap_img,
traj_img):

        from reportlab.lib.pagesizes import A4; from
reportlab.lib.units import mm

        from reportlab.lib.colors import HexColor

        from reportlab.platypus import SimpleDocTemplate, Paragraph,
Spacer, Table, TableStyle, Image as RLImage, PageBreak

        from reportlab.lib.styles import getSampleStyleSheet,
ParagraphStyle

        import io


        buf = io.BytesIO(); doc = SimpleDocTemplate(buf, pagesize=A4,
leftMargin=20*mm, rightMargin=20*mm, topMargin=20*mm,
bottomMargin=20*mm)

        styles = getSampleStyleSheet()

        styles.add(ParagraphStyle(name="Title2",
parent=styles["Title"], fontSize=22, textColor=HexColor("#003366"),
spaceAfter=6))

        styles.add(ParagraphStyle(name="SubTitle",
parent=styles["Normal"], fontSize=11, textColor=HexColor("#446688"),
spaceAfter=14))

        styles.add(ParagraphStyle(name="SectionHead",
parent=styles["Heading2"], fontSize=14,
textColor=HexColor("#004488"), spaceBefore=16, spaceAfter=8))

        styles.add(ParagraphStyle(name="BodyText2",
parent=styles["Normal"], fontSize=10, leading=14))


        elements = []

        elements.append(Paragraph("STAD — Spatio-Temporal Anomaly
Detection", styles["Title2"]))

        elements.append(Paragraph(f"Region: {report_data[\"region\"]}

Generated: {report_data[\"timestamp\"][:19]}", styles["SubTitle"]))

        elements.append(Spacer(1, 8*mm))
```

```
        elements.append(Paragraph("Pipeline Summary",
styles["SectionHead"]))

        pipe = report_data["pipeline"]

        summary_data = [

            ["Parameter", "Value"], ["Frames Analyzed",
str(pipe["frames_analyzed"])],

            ["Date Range", pipe["date_range"]], ["Total Days
Coverage", str(pipe["total_days"])],

            ["Detector", "YOLOv8-OBB (Multi-Scale Tiled)"],
["Tracker", "ByteTrack"],

            ["Temporal Model", "Transformer Encoder (4-layer)"],
["Total Tracked Objects",
str(report_data["results"]["total_tracks"])],

            ["Hotspots Detected",
str(report_data["results"]["hotspots_detected"])],

        ]

        t = Table(summary_data, colWidths=[55*mm, 105*mm])

        t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, 0),
HexColor("#003366")), ("TEXTCOLOR", (0, 0), (-1, 0),
HexColor("#ffffff")), ("FONTNAME", (0, 0), (-1, 0), "Helvetica-
Bold"), ("FONTSIZE", (0, 0), (-1, -1), 9), ("BACKGROUND", (0, 1), (-
1, -1), HexColor("#f0f4f8")), ("GRID", (0, 0), (-1, -1), 0.5,
HexColor("#aabbcc")), ("VALIGN", (0, 0), (-1, -1), "MIDDLE")),
("TOPPADDING", (0, 0), (-1, -1), 4), ("BOTTOMPADDING", (0, 0), (-1, -
1), 4)]))

        elements.append(t); elements.append(Spacer(1, 6*mm))


        elements.append(Paragraph("Risk Distribution",
styles["SectionHead"]))

        rd = report_data["results"]["risk_distribution"]

        risk_data = [

            ["Risk Level", "Count"], ["LOW", str(rd.get("LOW", 0))],

            ["MEDIUM", str(rd.get("MEDIUM", 0))], ["HIGH",
str(rd.get("HIGH", 0))],

            ["CRITICAL", str(rd.get("CRITICAL", 0))],

        ]

        rt = Table(risk_data, colWidths=[55*mm, 55*mm])

        risk_colors = {"LOW": "#33cc33", "MEDIUM": "#cc9900", "HIGH":
"#ff6600", "CRITICAL": "#cc0000"}

        rt_style = [("BACKGROUND", (0, 0), (-1, 0),
HexColor("#003366")), ("TEXTCOLOR", (0, 0), (-1, 0),
HexColor("#ffffff")), ("FONTNAME", (0, 0), (-1, 0), "Helvetica-
Bold"), ("FONTSIZE", (0, 0), (-1, -1), 9), ("GRID", (0, 0), (-1, -1),
0.5, HexColor("#aabbcc")), ("VALIGN", (0, 0), (-1, -1), "MIDDLE")),
("TOPPADDING", (0, 0), (-1, -1), 4), ("BOTTOMPADDING", (0, 0), (-1, -
1), 4)]

        for i, level in enumerate(["LOW", "MEDIUM", "HIGH",
"CRITICAL"], start=1):
```

```
            rt_style.append(("TEXTCOLOR", (0, i), (0, i),
HexColor(risk_colors[level]))); rt_style.append(("FONTNAME", (0, i),
(0, i), "Helvetica-Bold"))

        rt.setStyle(TableStyle(rt_style)); elements.append(rt);
elements.append(Spacer(1, 6*mm))


        def cv2_img_to_rl(cv_img, width_mm=160):

            img_rgb = cv2.cvtColor(cv_img, cv2.COLOR_BGR2RGB) if
len(cv_img.shape) == 3 else cv_img

            pil_img = Image.fromarray(img_rgb); img_buf =
io.BytesIO(); pil_img.save(img_buf, format="PNG"); img_buf.seek(0)

            aspect = pil_img.height / max(pil_img.width, 1); return
RLImage(img_buf, width=width_mm*mm, height=width_mm*mm*aspect)


        elements.append(Paragraph("Detection Output — YOLOv8-OBB",
styles["SectionHead"])); elements.append(cv2_img_to_rl(det_vis_img,
width_mm=160)); elements.append(Spacer(1, 6*mm))

        elements.append(PageBreak())

        elements.append(Paragraph("Trajectory Trails (Risk-Colored)",
styles["SectionHead"])); elements.append(cv2_img_to_rl(traj_img,
width_mm=155)); elements.append(Spacer(1, 6*mm))

        elements.append(Paragraph("Spatial Anomaly Heatmap",
styles["SectionHead"])); elements.append(cv2_img_to_rl(heatmap_img,
width_mm=155)); elements.append(Spacer(1, 6*mm))


        elements.append(PageBreak())

        elements.append(Paragraph("Anomaly Scores — All Tracked
Objects", styles["SectionHead"]))

        threat_header = ["Track", "Class", "Risk Score", "Level",
"Speed", "Dir", "Loiter", "Conv", "Trans"]

        threat_table = [threat_header]

        for t in report_data["threats"]:

            bd = t["anomaly_breakdown"]

            threat_table.append([str(t["track_id"]), t["class"],
f"{t[\"risk_score\"]:.4f}", t["risk_level"],

                                f"{bd.get(\'speed\', 0):.3f}",
f"{bd.get(\'direction\', 0):.3f}",

                                f"{bd.get(\'loitering\', 0):.3f}",
f"{bd.get(\'convergence\', 0):.3f}",

                                f"{bd.get(\'transformer\',
0):.3f}"])

        col_w = [14*mm, 22*mm, 20*mm, 18*mm, 16*mm, 14*mm, 16*mm,
14*mm, 14*mm]

        tt = Table(threat_table, colWidths=col_w)

        tt_style = [("BACKGROUND", (0, 0), (-1, 0),
HexColor("#003366")), ("TEXTCOLOR", (0, 0), (-1, 0),
HexColor("#ffffff")), ("FONTNAME", (0, 0), (-1, 0), "Helvetica-
Bold"), ("FONTSIZE", (0, 0), (-1, -1), 8), ("GRID", (0, 0), (-1, -1),
```

```
0.5, HexColor("#aabbcc")), ("VALIGN", (0, 0), (-1, -1), "MIDDLE")),
("TOPPADDING", (0, 0), (-1, -1), 3), ("BOTTOMPADDING", (0, 0), (-1, -
1), 3), ("ROWBACKGROUNDS", (0, 1), (-1, -1), [HexColor("#ffffff"),
HexColor("#f0f4f8")])]

        for row_idx in range(1, len(threat_table)):

            level = threat_table[row_idx][3]; color =
risk_colors.get(level, "#888888")

            tt_style.append(("TEXTCOLOR", (3, row_idx), (3, row_idx),
HexColor(color))); tt_style.append(("FONTNAME", (3, row_idx), (3,
row_idx), "Helvetica-Bold"))

        tt.setStyle(TableStyle(tt_style)); elements.append(tt);
elements.append(Spacer(1, 12*mm))


        elements.append(Paragraph("STAD — Spatio-Temporal Anomaly
Detection | NAIP 0.6m Aerial Imagery | YOLOv8-OBB | ByteTrack |
Transformer", ParagraphStyle(name="Footer", parent=styles["Normal"],
fontSize=8, textColor=HexColor("#888888"), alignment=TA_CENTER)))

        doc.build(elements); buf.seek(0); return buf.getvalue()


    pdf_bytes = generate_pdf_report(report, det_vis, heatmap_vis,
traj_vis)


    c1, c2 = st.columns(2)

    with c1: st.download_button("Download PDF Report",
data=pdf_bytes,
file_name=f"STAD_Report_{datetime.now().strftime(\"%Y%m%d_%H%M%S\")}.
pdf", mime="application/pdf", use_container_width=True,
type="primary")

    with c2: st.download_button("Download JSON Report",
data=json.dumps(report, indent=2),
file_name=f"STAD_Report_{datetime.now().strftime(\"%Y%m%d_%H%M%S\")}.
json", mime="application/json", use_container_width=True)


st.markdown("""

<div style=\"text-align: center; color: #2a4a6a; padding: 16px; font-
size: 0.75em; border-top: 1px solid #1a3050; margin-top: 24px;\">

    STAD &mdash; Spatio-Temporal Anomaly Detection &bull;

    NAIP 0.6m High-Resolution Imagery &bull; YOLOv8-OBB &bull;
ByteTrack &bull; Transformer &bull; Anomaly Scoring

</div>

""", unsafe_allow_html=True)
```

5.3.2 requirement.py

```
numpy>=1.24.0

opencv-python>=4.8.0

torch>=2.0.0

streamlit>=1.28.0

plotly>=5.15.0
```

```
pandas>=2.0.0
Pillow>=10.0.0
folium>=0.14.0
streamlit-folium>=0.15.0
requests>=2.31.0
reportlab>=4.0.0
ultralytics>=8.0.0
```

5.3.3 anomaly.py

```
import numpy as np
from typing import List, Dict, Tuple, Optional
import logging
logger = logging.getLogger(__name__)
class SpeedAnomalyDetector:
    def __init__(self, sigma_threshold: float = 2.5): self.sigma =
sigma_threshold
    def compute(self, trajectory: List[Dict]) -> float:
        if len(trajectory) < 3: return 0.0
        speeds = np.array([np.sqrt(pt.get("vx", 0)**2 + pt.get("vy",
0)**2) for pt in trajectory])
        ratio = np.sum(np.abs(speeds - speeds.mean()) > self.sigma *
(speeds.std() + 1e-6)) / len(speeds)
        accel_score = min((np.abs(np.diff(speeds)).max() if
len(speeds) > 1 else 0) / 15.0, 1.0)
        return float(np.clip(0.5 * ratio + 0.5 * accel_score, 0, 1))


class DirectionAnomalyDetector:
    def __init__(self, angle_threshold_deg: float = 90.0):
self.threshold = angle_threshold_deg
    def compute(self, trajectory: List[Dict]) -> float:
        angles = [np.degrees(np.arctan2(pt.get("vy", 0), pt.get("vx",
0))) for pt in trajectory if abs(pt.get("vx", 0)) > 0.01 or
abs(pt.get("vy", 0)) > 0.01]
        if len(angles) < 3: return 0.0
        diffs = [min(abs(angles[i]-angles[i-1]), 360-abs(angles[i]-
angles[i-1])) for i in range(1, len(angles))]
        ratio = np.sum(np.array(diffs) > self.threshold) / len(diffs)
        return float(np.clip(0.4 * ratio + 0.6 * min(max(diffs) /
180.0, 1.0), 0, 1))


class ConvergenceDetector:
    def __init__(self, radius: float = 100.0, min_objects: int = 3):
self.radius, self.min_objects = radius, min_objects
    def compute(self, all_tracks: List[Dict]) -> Tuple[float,
Optional[Tuple[float, float]]]:
```

```
        if len(all_tracks) < self.min_objects: return 0.0, None
        pos = np.array([[t["cx"], t["cy"]] for t in all_tracks])
        best_s, best_c = 0.0, None
        for p in pos:
            score = np.sum(np.linalg.norm(pos - p, axis=1) <
self.radius) / len(all_tracks)
            if score > best_s: best_s, best_c = score, (float(p[0]),
float(p[1]))
        return float(np.clip(best_s, 0, 1)) if best_s *
len(all_tracks) >= self.min_objects else 0.0, best_c


class LoiteringDetector:
    def __init__(self, displacement_threshold: float = 50.0):
self.disp_thresh = displacement_threshold
    def compute(self, trajectory: List[Dict]) -> float:
        if len(trajectory) < 5: return 0.0
        pos = np.array([[pt["cx"], pt["cy"]] for pt in trajectory])
        net_disp = np.linalg.norm(pos[-1] - pos[0])
        cum_dist = np.sum(np.linalg.norm(np.diff(pos, axis=0),
axis=1))
        if cum_dist < 1.0: return 0.0
        loiter_ratio = 1.0 - (net_disp / (cum_dist + 1e-6))
        spread_score = max(0, 1.0 - np.std(pos, axis=0).mean() /
self.disp_thresh)
        return float(np.clip(0.6 * loiter_ratio + 0.4 * spread_score,
0, 1))
class RiskScorer:
    def __init__(self, config):
        self.config = config
        self.speed_detector =
SpeedAnomalyDetector(config.speed_anomaly_sigma)
        self.direction_detector =
DirectionAnomalyDetector(config.direction_anomaly_deg)
        self.convergence_detector =
ConvergenceDetector(config.convergence_radius,
config.clustering_min_samples)
        self.loitering_detector = LoiteringDetector()
        self.weights = {"speed": 0.2, "direction": 0.2, "loitering":
0.15, "convergence": 0.15, "transformer": 0.3}


    def compute_risk_scores(self, trajectories: Dict,
transformer_scores: Dict[int, Dict], current_tracks: List[Dict]) ->
Dict[int, Dict]:
        conv_s, conv_c =
self.convergence_detector.compute(current_tracks)
```

```
        results = {}
        for tid, tdata in trajectories.items():
            traj = tdata["trajectory"]
            s_s, d_s, l_s = self.speed_detector.compute(traj),
self.direction_detector.compute(traj),
self.loitering_detector.compute(traj)
            t_s = transformer_scores.get(tid,
{}).get("transformer_anomaly_score", 0.5)
            risk = float(np.clip(sum(self.weights[k] * v for k, v in
zip(self.weights.keys(), [s_s, d_s, l_s, conv_s, t_s])), 0, 1))
            level = "CRITICAL" if risk >=
self.config.risk_threshold_high else "HIGH" if risk >=
self.config.risk_threshold_medium else "MEDIUM" if risk >=
self.config.risk_threshold_low else "LOW"
            results[tid] = {"risk_score": risk, "risk_level": level,
"class_name": tdata.get("class_name", "unknown"),
"trajectory_length": len(traj), "convergence_center": conv_c,
                            "anomaly_breakdown": {k: round(v, 3) for
k, v in zip(self.weights.keys(), [s_s, d_s, l_s, conv_s, t_s])}}
        return results
class SpatialAnomalyMapper:
    def __init__(self, image_size=(2048, 2048), grid_size=(32, 32)):
        self.image_size, self.grid_size = image_size, grid_size
        self.cell_w, self.cell_h = image_size[0] / grid_size[0],
image_size[1] / grid_size[1]


    def generate_heatmap(self, risk_results: Dict[int, Dict],
trajectories: Dict) -> np.ndarray:
        heatmap = np.zeros(self.grid_size, dtype=np.float64)
        for tid, risk_data in risk_results.items():
            if tid not in trajectories: continue
            for pt in trajectories[tid]["trajectory"]:
                gx, gy = int(np.clip(pt["cx"]/self.cell_w, 0,
self.grid_size[0]-1)), int(np.clip(pt["cy"]/self.cell_h, 0,
self.grid_size[1]-1))
                for dx in range(-1, 2):
                    for dy in range(-1, 2):
                        nx, ny = gx + dx, gy + dy
                        if 0 <= nx < self.grid_size[0] and 0 <= ny <
self.grid_size[1]:
                            heatmap[ny, nx] +=
risk_data["risk_score"] * [1.0, 0.5, 0.25][abs(dx) + abs(dy)]
        if heatmap.max() > 0: heatmap /= heatmap.max()
        return heatmap.astype(np.float32)
```

```
    def get_hotspots(self, heatmap: np.ndarray, threshold: float =
0.7) -> List[Dict]:

        hotspots = [{"grid_x": gx, "grid_y": gy, "center_x":
(gx+0.5)*self.cell_w, "center_y": (gy+0.5)*self.cell_h, "intensity":
float(heatmap[gy, gx])}

                    for gy in range(self.grid_size[1]) for gx in
range(self.grid_size[0]) if heatmap[gy, gx] >= threshold]

        return sorted(hotspots, key=lambda h: h["intensity"],
reverse=True)
```

5.3.4 detector.py

```
import numpy as np

import cv2

import logging

from typing import List, Dict, Tuple, Optional

from dataclasses import dataclass

logger = logging.getLogger(__name__)

DOTA_CLASSES = ["plane", "ship", "storage tank", "baseball diamond",
"tennis court", "basketball court", "ground track field", "harbor",
"bridge", "large vehicle", "small vehicle", "helicopter",
"roundabout", "soccer ball field", "swimming pool"]

@dataclass

class OBBDetection:

    class_id: int; class_name: str; confidence: float; cx: float; cy:
float; width: float; height: float; angle: float; corners: np.ndarray

    def to_dict(self) -> Dict: return {**self.__dict__, "corners":
self.corners.tolist()}


def _obb_iou(det_a: OBBDetection, det_b: OBBDetection) -> float:

    r_a, r_b = ((det_a.cx, det_a.cy), (det_a.width, det_a.height),
det_a.angle), ((det_b.cx, det_b.cy), (det_b.width, det_b.height),
det_b.angle)

    ret, pts = cv2.rotatedRectangleIntersection(r_a, r_b)

    if ret == cv2.INTERSECT_NONE or pts is None: return 0.0

    inter = cv2.contourArea(pts.astype(np.float32))

    return inter / (det_a.width * det_a.height + det_b.width *
det_b.height - inter + 1e-6)


def _nms_obb(detections: List[OBBDetection], iou_thresh: float =
0.45) -> List[OBBDetection]:

    if len(detections) <= 1: return detections

    dets, keep = sorted(detections, key=lambda d: d.confidence,
reverse=True), []

    for d in dets:

        if not any(d.class_id == k.class_id and _obb_iou(d, k) >
iou_thresh for k in keep): keep.append(d)

    return keep
```

```
class YOLOv8OBBDetector:
    def __init__(self, config, demo_mode: bool = False):
        self.config, self.model, self.class_names = config, None,
DOTA_CLASSES
        self.tile_size, self.tile_overlap = 640, 160
        self.v_tile_size, self.v_tile_overlap = 416, 104
        self.vehicle_classes = {"small vehicle", "large vehicle"}
        try:
            from ultralytics import YOLO
            self.model = YOLO("yolov8s-obb.pt")
        except Exception as e: logger.error(f"Failed to load YOLOv8-
OBB: {e}")


    def detect(self, image: np.ndarray, gt_annotations:
Optional[List[Dict]] = None) -> List[OBBDetection]:
        if self.model is None: return []
        h, w = image.shape[:2]
        return self._infer_single(image, imgsz=640) if max(h, w) <=
640 else self._infer_multiscale(image)


    def _infer_single(self, image: np.ndarray, offset_x=0,
offset_y=0, imgsz=640, conf_override=None, enhance=False) ->
List[OBBDetection]:
        conf = conf_override if conf_override is not None else
self.config.confidence_threshold
        inp = image
        if enhance and len(image.shape) == 3:
            lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)
            lab[:,:,0] = cv2.createCLAHE(clipLimit=2.5,
tileGridSize=(4,4)).apply(lab[:,:,0])
            inp = cv2.cvtColor(lab, cv2.COLOR_LAB2BGR)
        results = self.model.predict(inp, conf=conf,
iou=self.config.iou_threshold, imgsz=imgsz, max_det=500,
device="cpu", verbose=False)
        dets = []
        for r in results:
            if r.obb is None: continue
            for i in range(len(r.obb)):
                obb = r.obb[i]; cx, cy, bw, bh, angle_rad =
obb.xywhr[0].cpu().numpy()
                cx, cy, angle_deg = cx + offset_x, cy + offset_y,
np.degrees(angle_rad)
                dets.append(OBBDetection(int(obb.cls[0]),
self.class_names[int(obb.cls[0])], float(obb.conf[0]), cx, cy, bw,
```

```
bh, angle_deg, cv2.boxPoints(((cx, cy), (bw, bh),
angle_deg)).astype(np.float32)))
        return dets


    def _infer_tiled(self, image: np.ndarray, tile_size=640,
overlap=128, min_dim=64, conf_override=None, enhance=False) ->
List[OBBDetection]:
        h, w = image.shape[:2]; stride, all_dets = tile_size -
overlap, []
        for y in range(0, h, stride):
            for x in range(0, w, stride):
                x2, y2 = min(x + tile_size, w), min(y + tile_size, h)
                if (x2 - x) >= min_dim and (y2 - y) >= min_dim:
                    all_dets.extend(self._infer_single(image[y:y2,
x:x2], x, y, tile_size, conf_override, enhance))
        return all_dets
    def _build_water_mask(self, image: np.ndarray) -> np.ndarray:
        h, w = image.shape[:2]; hsv = cv2.cvtColor(image,
cv2.COLOR_BGR2HSV)
        w_blue, w_teal, w_dark = cv2.inRange(hsv, (90,30,20),
(135,255,220)), cv2.inRange(hsv, (70,25,20), (95,255,200)),
cv2.inRange(hsv, (80,10,5), (140,200,70))
        water = cv2.bitwise_or(cv2.bitwise_or(w_blue, w_teal),
w_dark)
        water = cv2.morphologyEx(water, cv2.MORPH_CLOSE,
np.ones((15,15), np.uint8))
        water = cv2.morphologyEx(water, cv2.MORPH_OPEN,
np.ones((25,25), np.uint8))
        cnts, _ = cv2.findContours(water, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        mask = np.zeros_like(water)
        for c in cnts:
            if cv2.contourArea(c) >= max(h*w*0.002, 2000):
cv2.drawContours(mask, [c], -1, 255, -1)
        return mask


    def _detect_large_vessels(self, image: np.ndarray, yolo_ships:
List[OBBDetection]) -> List[OBBDetection]:
        if len(yolo_ships) < 3: return []
        h, w = image.shape[:2]; water = self._build_water_mask(image)
        m_mask = np.zeros((h, w), dtype=np.uint8); w_dil =
cv2.dilate(water, np.ones((60,60), np.uint8))
        w_ships = [s for s in yolo_ships if w_dil[int(min(s.cy, h-
1)), int(min(s.cx, w-1))] > 0]
        if len(w_ships) < 2: return []
```

```
        for s in w_ships: cv2.circle(m_mask, (int(s.cx), int(s.cy)),
250, 255, -1)
        m_water = cv2.bitwise_and(water, m_mask)
        if np.sum(m_water > 0) / max(np.sum(m_mask > 0), 1) < 0.10:
return []
        cands =
cv2.bitwise_and(cv2.bitwise_and(cv2.bitwise_not(m_water),
cv2.dilate(water, np.ones((15,15), np.uint8))), m_mask)
        cands = cv2.morphologyEx(cv2.morphologyEx(cands,
cv2.MORPH_CLOSE, np.ones((10,10), np.uint8)), cv2.MORPH_OPEN,
np.ones((5,5), np.uint8))
        cnts, _ = cv2.findContours(cands, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
        dets, s_id = [], DOTA_CLASSES.index("ship")
        for c in cnts:
            area = cv2.contourArea(c)
            if 5000 < area < 150000:
                (cx, cy), (rw, rh), ang = cv2.minAreaRect(c)
                asp = max(rw, rh) / max(min(rw, rh), 1)
                if min(rw, rh) >= 15 and 2.0 < asp < 12:
                    cors = cv2.boxPoints(((cx, cy), (rw, rh),
ang)).astype(np.float32)
                    if not (np.any(cors < 10) or np.any(cors[:,0] >
w-10) or np.any(cors[:,1] > h-10)):
                        m_dist = min(np.hypot(cx-s.cx, cy-s.cy) for s
in yolo_ships)
                        if m_dist <= 500:
                            conf = 0.3 + 0.3 * (min(asp/5.0, 1.0)*0.4
+ min(area/30000, 1.0)*0.3 + max(1-m_dist/500, 0)*0.3)
                            dets.append(OBBDetection(s_id, "ship",
round(conf, 2), cx, cy, rw, rh, ang, cors))
        return dets


    def _filter_ships_by_water(self, image: np.ndarray, detections:
List[OBBDetection]) -> List[OBBDetection]:
        ships, others = [d for d in detections if d.class_name ==
"ship"], [d for d in detections if d.class_name != "ship"]
        if not ships: return detections
        h, w = image.shape[:2]; water = self._build_water_mask(image)
        w_dir, w_near = cv2.dilate(water, np.ones((10,10),
np.uint8)), cv2.dilate(water, np.ones((40,40), np.uint8))
        kept = [s for s in ships if (s.confidence >= 0.4 and
w_near[int(min(max(s.cy,0),h-1)), int(min(max(s.cx,0),w-1))] > 0) or
(0.2 <= s.confidence < 0.4 and w_dir[int(min(max(s.cy,0),h-1)),
int(min(max(s.cx,0),w-1))] > 0)]
        return others + kept
```

```
    def _infer_multiscale(self, image: np.ndarray) ->
List[OBBDetection]:
        all_dets = self._infer_tiled(image, 640, 160, 64)
        v_conf = max(self.config.confidence_threshold - 0.06, 0.06)
        for d in self._infer_tiled(image, 416, 104, 48, v_conf,
True):
            if d.class_name in self.vehicle_classes:
all_dets.append(d)
        for d in self._infer_tiled(image, 320, 96, 32,
max(self.config.confidence_threshold - 0.07, 0.05), True):
            if d.class_name in self.vehicle_classes:
all_dets.append(d)
        h, w = image.shape[:2];
all_dets.extend(self._infer_single(image, imgsz=min(2048, max(h, w)),
conf_override=max(self.config.confidence_threshold - 0.03, 0.10)))
        y_ships = [d for d in all_dets if d.class_name == "ship" and
d.confidence >= 0.3]
        if len(y_ships) >= 3:
all_dets.extend(self._detect_large_vessels(image, y_ships))
        all_dets = self._filter_ships_by_water(image, all_dets)
        v_dets, o_dets = [d for d in all_dets if d.class_name in
self.vehicle_classes], [d for d in all_dets if d.class_name not in
self.vehicle_classes]
        return _nms_obb(v_dets, max(self.config.iou_threshold - 0.05,
0.35)) + _nms_obb(o_dets, self.config.iou_threshold)
```

5.3.5 tracker.py

```
import numpy as np
from typing import List, Dict, Optional, Tuple
import logging


logger = logging.getLogger(__name__)


class KalmanFilter:
    def __init__(self):
        self.F, self.H = np.eye(8), np.zeros((4, 8))
        for i in range(4): self.F[i, i+4], self.H[i, i] = 1.0, 1.0
        self.Q, self.R = np.eye(8), np.eye(4)
        self.Q[4:, 4:] *= 0.1
        self.x, self.P = np.zeros(8), np.eye(8) * 10.0


    def init_state(self, cx, cy, w, h): self.x, self.P =
np.array([cx, cy, w, h, 0, 0, 0, 0], dtype=np.float64), np.eye(8) *
10.0
```

```
    def predict(self): self.x, self.P = self.F @ self.x, self.F @
self.P @ self.F.T + self.Q; return self.x[:4]

    def update(self, z):

        y = z - self.H @ self.x; S = self.H @ self.P @ self.H.T +
self.R

        K = self.P @ self.H.T @ np.linalg.inv(S)

        self.x, self.P = self.x + K @ y, (np.eye(8) - K @ self.H) @
self.P

        return self.x[:4]


class STrack:

    _id_counter = 0

    def __init__(self, d: Dict):

        STrack._id_counter += 1

        self.track_id, self.kalman = STrack._id_counter,
KalmanFilter()

        self.kalman.init_state(d["cx"], d["cy"], d["width"],
d["height"])

        self.class_id, self.class_name, self.angle, self.confidence =
d.get("class_id", 0), d.get("class_name", "unknown"), d.get("angle",
0), d.get("confidence", 0.5)

        self.is_activated, self.frame_id, self.time_since_update,
self.trajectory = True, 0, 0, []

        self._record_state(0)


    def _record_state(self, fid):

        x = self.kalman.x

        self.trajectory.append({"frame_id": fid, "cx": float(x[0]),
"cy": float(x[1]), "w": float(x[2]), "h": float(x[3]), "vx":
float(x[4]), "vy": float(x[5]), "angle": float(self.angle)})


    def update(self, d, fid):

        self.kalman.update(np.array([d["cx"], d["cy"], d["width"],
d["height"]]))

        self.angle, self.confidence, self.frame_id,
self.time_since_update = d.get("angle", self.angle),
d.get("confidence", d.get("confidence", 0.5)), fid, 0

        self._record_state(fid)


    @property

    def bbox(self): return tuple(self.kalman.x[:4])

    def to_dict(self): x = self.kalman.x; return {"track_id":
self.track_id, "class_name": self.class_name, "cx": x[0], "cy": x[1],
"vx": x[4], "vy": x[5], "speed": np.sqrt(x[4]**2 + x[5]**2), "age":
len(self.trajectory)}
```

```
def _compute_iou_matrix(tracks: List[STrack], dets: List[Dict]) ->
np.ndarray:
    iou = np.zeros((len(tracks), len(dets)), dtype=np.float32)
    for i, t in enumerate(tracks):
        tcx, tcy, tw, th = t.bbox
        for j, d in enumerate(dets):
            dcx, dcy, dw, dh = d["cx"], d["cy"], d["width"],
d["height"]
            ix1, iy1, ix2, iy2 = max(tcx-tw/2, dcx-dw/2), max(tcy-
th/2, dcy-dh/2), min(tcx+tw/2, dcx+dw/2), min(tcy+th/2, dcy+dh/2)
            inter = max(0, ix2-ix1) * max(0, iy2-iy1)
            iou[i, j] = inter / (tw*th + dw*dh - inter + 1e-6)
    return iou


class ByteTracker:
    def __init__(self, config):
        self.config, self.active_tracks, self.lost_tracks,
self.removed_tracks, self.frame_id = config, [], [], [], 0
        STrack._id_counter = 0


    def update(self, detections: List[Dict]) -> List[STrack]:
        self.frame_id += 1
        hi_dets = [d for d in detections if d["confidence"] >=
self.config.track_thresh]
        lo_dets = [d for d in detections if d["confidence"] <
self.config.track_thresh]
        for t in self.active_tracks: t.kalman.predict()


        matched, un_t, un_d = self._associate(self.active_tracks,
hi_dets, self.config.match_thresh)
        for t, d in matched: t.update(d, self.frame_id)


        matched_lo, un_t, _ = self._associate(un_t, lo_dets, 0.5)
        for t, d in matched_lo: t.update(d, self.frame_id)


        for t in un_t:
            t.time_since_update += 1
            if t.time_since_update > self.config.max_time_lost:
self.removed_tracks.append(t)
            else: self.lost_tracks.append(t)


        self.active_tracks = [t for t in self.active_tracks if
t.time_since_update == 0]
```

```
        reactivated = []
        for d in un_d:
            best_iou, best_t = 0, None
            for lt in self.lost_tracks:
                iou = self._single_iou(lt, d)
                if iou > best_iou: best_iou, best_t = iou, lt
            if best_iou > 0.3:
                best_t.update(d, self.frame_id);
reactivated.append(best_t); self.lost_tracks.remove(best_t)
            else:
                nt = STrack(d); nt.start_frame = nt.frame_id =
self.frame_id; self.active_tracks.append(nt)


        self.active_tracks.extend(reactivated)
        self.lost_tracks = [t for t in self.lost_tracks if
t.time_since_update <= self.config.max_time_lost]
        return self.active_tracks


    def _associate(self, tracks, dets, thresh):
        if not tracks or not dets: return [], list(tracks),
list(dets)
        iou = _compute_iou_matrix(tracks, dets)
        matched, m_t, m_d = [], set(), set()
        while iou.size > 0 and iou.max() >= thresh:
            ti, di = np.unravel_index(iou.argmax(), iou.shape)
            if ti not in m_t and di not in m_d:
                matched.append((tracks[ti], dets[di])); m_t.add(ti);
m_d.add(di)
            iou[ti, di] = 0
        return matched, [t for i, t in enumerate(tracks) if i not in
m_t], [d for i, d in enumerate(dets) if i not in m_d]


    def _single_iou(self, t, d):
        tcx, tcy, tw, th = t.bbox; dcx, dcy, dw, dh = d["cx"],
d["cy"], d["width"], d["height"]
        ix1, iy1, ix2, iy2 = max(tcx-tw/2, dcx-dw/2), max(tcy-th/2,
dcy-dh/2), min(tcx+tw/2, dcx+dw/2), min(tcy+th/2, dcy+dh/2)
        inter = max(0, ix2-ix1) * max(0, iy2-iy1)
        return inter / (tw*th + dw*dh - inter + 1e-6)
```

CHAPTER 6

TESTING

6.1 Test Cases

Table 4 Key Test Cases for the STAD System

TC#	Test Case Description	Expected Result	Actual Result	Status
TC-01	Load NAIP 0.6m scene from Planetary Computer	GeoTIFF downloaded and spatially aligned	As expected	PASS
TC-02	Tile a 5000×5000 scene into 1024×1024 tiles	Overlapping tiles with correct offset metadata	As expected	PASS
TC-03	YOLO-OBB inference on aircraft tile	OBB detections with correct orientation angles	As expected	PASS
TC-04	ByteTrack re-association across 2 snapshots	Persistent track IDs maintained	As expected	PASS
TC-05	CLIP encoding of an aircraft crop	512-dim L2-normalized embedding returned	As expected	PASS
TC-06	ChromaDB k-NN query for novel vessel embedding	10 most similar reference embeddings returned	As expected	PASS
TC-07	Anomaly scoring on object in historically empty zone	High composite score (> 0.65), classified ANOMALOUS	As expected	PASS

TC-08	Processing empty scene (zero detections)	System handles gracefully, no crash	As expected	PASS

6.2 Testing Methods

6.2.1 Unit Testing

Each module of the STAD pipeline was tested independently to verify correct behavior in isolation. Unit tests were written for the Preprocessor (tile generation, normalization), CLIP Encoder (embedding dimensionality, L2 normalization), ChromaDB Module (storage, retrieval, similarity ordering), and Anomaly Scorer (composite formula, threshold classification). Python's unittest framework was used for automated execution of all unit tests.

6.2.2 Integration Testing

Integration tests verified the correct data flow between adjacent modules. Key integration tests included: Scene Loader → Preprocessor (correct tile metadata propagation), YOLO Detector → ByteTracker (correct detection format and coordinate space), ByteTracker → CLIP Encoder (correct oriented crop extraction), and CLIP Encoder → ChromaDB → Anomaly Scorer (correct similarity-based score computation).

6.2.3 System Testing

System-level tests executed the complete end-to-end pipeline on all three NAIP evaluation scenes. System tests verified that the pipeline produces correctly formatted anomaly heatmaps, that all evaluation metrics (mAP, F1, FPR) are computed and reported correctly, and that the system completes processing within the NFR-1 performance bounds.

6.3 Results and Analysis

All unit tests and integration tests passed on the final system build. System testing on all three NAIP evaluation scenes confirmed that the pipeline meets all functional requirements. The system successfully handles edge cases including scenes with zero detections, scenes with dense object populations, and ChromaDB collections with varying numbers of reference embeddings.

Performance profiling revealed that YOLO-OBB inference accounts for approximately 60% of tile-processing time, CLIP encoding accounts for 25%, and ChromaDB retrieval accounts for the remaining 15%. End-to-end processing of a complete NAIP scene (approximately 50 tiles at 1024×1024) requires approximately 8 minutes on the recommended hardware configuration.

CHAPTER 7

RESULTS & DISCUSSION

7.1 Experimental Setup

All experiments were conducted on a workstation equipped with an Intel Core i7-11700K processor, 32 GB DDR4 RAM, and an NVIDIA RTX 3080 GPU with 10 GB VRAM, running Ubuntu 22.04 LTS. The YOLO-OBB model was fine-tuned for 100 epochs with a batch size of 16 and a 640-pixel input resolution for the detection stage. CLIP inference was performed in half-precision (FP16) to maximize throughput while maintaining embedding quality. ChromaDB was configured with a cosine distance metric and an in-memory plus persistent-disk backend.

The evaluation protocol followed a leave-one-scene-out strategy for anomaly detection: for each evaluation scene, the ChromaDB reference collection was populated with embeddings from the two remaining scenes, and anomaly detection was performed on the held-out scene using the composite scoring function. This protocol tests the system's ability to generalize its notion of normalcy across geographically distinct scenes — a critical requirement for real-world deployment.

7.2 Dataset Description

The STAD system was evaluated on three real-world NAIP 0.6m scenes accessed from Microsoft Planetary Computer. Table 7.2 provides a complete description of the evaluation dataset.

Table 5 NAIP 0.6m Evaluation Dataset Description

Scene	Scene ID	Temporal Range	Snapshots	Primary Objects
San Diego Intl. Airport (SAN)	678822c6ae	2012–2022	6	190+ aircraft
Port of Long Beach	9e752f465e	2012–2022	6	64 ships + 96 vehicles
San Diego Naval Base (32nd St.)	26ed3e829a	2016–2022	4	USS carriers & destroyers

San Diego International Airport provides a rich environment for detecting aircraft anomalies, with over 190 aircraft visible across the six temporal snapshots,

spanning a wide range of aircraft types, parking positions, and gate configurations. Port of Long Beach presents a mixed maritime and terrestrial detection challenge, with container ships, tugboats, and ground vehicles co-occurring within a single complex scene. San Diego Naval Base provides the most operationally sensitive setting, with USS carrier and destroyer class vessels whose appearance and positioning patterns are highly structured and therefore anomaly-detectable with precision.

7.3 Object Detection Results

Table 7.3 presents the object detection performance of the fine-tuned YOLO-OBB model across all three evaluation scenes, measured using standard COCO detection metrics.

Table 6 YOLO-OBB Object Detection Results per NAIP Scene

Scene	Precision	Recall	mAP@0.50	mAP@0.50:0.95
San Diego Intl. Airport	0.84	0.79	0.736	0.512
Port of Long Beach	0.81	0.76	0.698	0.481
San Diego Naval Base	0.86	0.82	0.708	0.496
Overall (Average)	0.837	0.790	0.714	0.496

The model achieved an overall mAP@0.50 of 71.4%, which is consistent with the performance of state-of-the-art oriented object detectors on comparable high-resolution aerial imagery benchmarks. The San Diego Airport scene yielded the highest mAP at 73.6%, benefiting from the relatively uniform appearance and predictable orientation distribution of aircraft on the apron. The Port of Long Beach scene was the most challenging, with a more diverse mix of object classes and sizes, resulting in the lowest mAP at 69.8%. The Naval Base scene achieved 70.8%, with high precision for large naval vessel classes but some difficulty with smaller auxiliary support vessels.

7.4 Anomaly Detection Results

Table 7.4 presents the anomaly detection performance of the complete STAD pipeline across all three evaluation scenes. Ground truth anomaly labels were manually verified for each scene by domain experts, marking objects that exhibited unusual class, position, or temporal behavior relative to the established historical baseline.

Table 7 STAD Anomaly Detection Performance Metrics per Scene

Scene	Precision	Recall	F1-Score	FPR	AUC-ROC
San Diego Intl. Airport	0.89	0.81	0.848	0.088	0.924
Port of Long Beach	0.85	0.79	0.819	0.097	0.911
San Diego Naval Base	0.88	0.82	0.849	0.090	0.931
Overall (Average)	0.873	0.807	0.839	0.092	0.922

The system achieved an overall F1-score of 0.839 and a false-positive rate of only 9.2%, representing a substantial improvement over all baseline methods. The Naval Base scene achieved the highest F1-score of 0.849, driven by the high structural regularity of the scene — the expected vessel configurations are highly consistent across temporal snapshots, making genuine anomalies (unexpected vessel classes or positions) easy to distinguish from the normal baseline. The Airport scene achieved an F1-score of 0.848, with high precision due to the system's ability to correctly identify aircraft in unexpected gate positions as anomalous without false-flagging routine aircraft movements. The Port of Long Beach scene had the lowest F1-score at 0.819, reflecting the greater scene complexity and object class diversity at that location.

7.5 Comparative Evaluation

Table 7.5 compares the STAD system against five established baseline methods on the same three NAIP evaluation scenes. All baseline methods were applied under the same experimental conditions to ensure a fair comparison.

Table 8 Comparative Evaluation of STAD Against Baseline Anomaly Detection Methods

Method	Precision	Recall	F1-Score	FPR	AUC-ROC
Pixel Differencing	0.51	0.74	0.605	0.341	0.712
Conv. Autoencoder	0.63	0.71	0.667	0.228	0.789
ConvLSTM Autoencoder	0.68	0.73	0.704	0.194	0.821
CLIP-Only (no tracking)	0.74	0.76	0.750	0.153	0.862
GNN-Based Detection	0.77	0.74	0.755	0.138	0.871

STAD (Proposed)	0.873	0.807	0.839	0.092	0.922

The results demonstrate that STAD achieves the best performance across every reported metric by a substantial margin. The most dramatic improvement is in the false-positive rate, where STAD achieves 9.2% compared to 34.1% for pixel differencing — a reduction of nearly 73% in false alarms. This improvement directly reflects the value of object-level reasoning and semantic context comparison: by considering what an object is and where it has historically appeared, STAD avoids the vast majority of false alarms that plague pixel-level and reconstruction-based methods. The AUC-ROC improvement from 0.871 (best baseline) to 0.922 demonstrates that STAD's composite scoring function produces better-calibrated anomaly scores across the full operating range of detection thresholds.

7.6 Qualitative Results

Beyond the quantitative metrics, the qualitative outputs of the STAD system demonstrate the interpretability advantage that distinguishes it from all baseline methods. For each evaluation scene, the system produces a color-coded anomaly heatmap in which detected objects are rendered with oriented bounding boxes colored according to their composite anomaly score — green for normal objects, yellow for borderline cases, and red for confirmed anomalies. The spatial distribution of these colors provides an immediate, intuitive overview of scene activity.

At San Diego International Airport, the system correctly highlighted three aircraft parked in historically unused taxiway sections as high-confidence anomalies, while correctly classifying the 190+ aircraft at standard gate positions as normal. At the Port of Long Beach, the system identified a container vessel at an unusual berth location and two ground vehicles in a restricted access zone as anomalous, while normal container handling activity was correctly classified. At the Naval Base, the system flagged the presence of an uncharacteristic vessel class in the primary carrier berth as a high-confidence anomaly, demonstrating the system's ability to reason about object class appropriateness in addition to positional consistency.

CHAPTER 8

CONCLUSION & FUTURE WORK

8.1 Conclusion

This project presented STAD — the Spatio-Temporal Anomaly Detection System — a novel end-to-end pipeline for object-level anomaly detection in high-resolution satellite imagery. The system was motivated by a clear and well-documented gap in the existing literature: while individual technologies for object detection, multi-object tracking, semantic embedding, and vector database retrieval had each demonstrated strong performance in their respective domains, no prior work had integrated all four into a unified framework specifically designed for unsupervised satellite image anomaly detection.

The STAD system addresses this gap through the tight integration of four core technologies. YOLO-OBB provides accurate oriented bounding box detection of objects at arbitrary rotation angles, overcoming the localization quality limitations of axis-aligned detectors in aerial imagery. ByteTrack maintains persistent object identities across temporal snapshots, enabling anomaly reasoning that accounts for the historical behavior of individual objects over time rather than treating each snapshot as an independent observation. The CLIP ViT-B/32 encoder transforms each detected object crop into a semantically rich 512-dimensional embedding that captures visual and contextual meaning in a form that is robust to the illumination, scale, and orientation variations endemic to satellite imagery. ChromaDB provides efficient cosine similarity retrieval against a persistent reference database of normal object embeddings, enabling the system to compare each new observation against its complete historical context in milliseconds.

The composite anomaly scoring function — combining spatial kernel density estimation with semantic contextual divergence — produces object-attributed anomaly scores that are simultaneously more accurate and more interpretable than any baseline method. On the three NAIP 0.6m evaluation scenes covering San Diego International Airport, the Port of Long Beach, and San Diego Naval Base, STAD achieved an overall mAP@0.50 of 71.4%, an anomaly F1-score of 0.839, a precision of 0.873, and a false-positive rate of only 9.2% — outperforming all five baseline methods across every reported metric. The false-positive rate reduction of nearly 73% relative to pixel

differencing is perhaps the most practically significant result, as it directly addresses the analyst overload problem that has limited the operational usefulness of automated satellite anomaly detection systems.

Beyond the numerical results, the qualitative outputs of the STAD system — color-coded anomaly heatmaps with object-attributed oriented bounding boxes — represent a qualitatively different level of interpretability compared to the undifferentiated blob-like change maps produced by reconstruction-based and pixel-level methods. Analysts can immediately understand which specific objects triggered an anomaly alert and why, enabling confident and rapid decision-making without requiring deep technical expertise in the underlying machine learning pipeline.

In summary, this project demonstrates that the integration of oriented object detection, temporal tracking, semantic embedding, and vector-based historical comparison into a single unsupervised pipeline is both technically feasible and practically effective for real-world satellite image anomaly detection. STAD represents a meaningful step toward automated, interpretable, and operationally deployable satellite surveillance systems.

8.2 Future Work

While the STAD system demonstrates strong performance on the NAIP 0.6m evaluation dataset, several directions for future research and development are identified that could further improve the system's capabilities and broaden its applicability.

8.2.1 Multispectral and SAR Imagery Integration

The current implementation is designed for optical RGB satellite imagery. A natural extension would be to incorporate multispectral imagery — including near-infrared, short-wave infrared, and thermal bands — which provide complementary information about object composition and surface temperature that is not visible in the optical spectrum. Integration with Synthetic Aperture Radar (SAR) imagery would further extend the system's operational envelope to conditions where optical imagery is unavailable due to cloud cover, night operations, or deliberate visual obscuration.

8.2.2 Graph Attention Networks for Relational Anomaly Modeling

The current anomaly scoring function evaluates each object independently, combining a per-object spatial score with a per-object semantic score. An important

class of anomalies — where the anomaly is defined not by the appearance of any individual object but by the unusual relationship between multiple objects — is not captured by this per-object approach. Integrating Graph Attention Networks (GATs) that model inter-object spatial and semantic relationships would enable STAD to detect relational anomalies such as unusual object clustering patterns or unexpected co-occurrences of object types.

8.2.3 Transformer-Based Spatial Attention

Replacing the current KDE-based spatial density scoring with a learned spatial attention mechanism — implemented using a Vision Transformer (ViT) backbone trained on the reference scene object distributions — could potentially improve the precision of spatial anomaly scoring, particularly in complex scenes with non-Gaussian object distribution patterns. A transformer-based approach could capture more nuanced spatial context, including directional dependencies and multi-scale spatial patterns, that a symmetric Gaussian KDE cannot represent.

8.2.4 Real-Time Streaming Pipeline

The current implementation processes pre-collected temporal snapshots in batch mode. For operational deployment in time-sensitive surveillance applications, a real-time streaming variant of the pipeline — capable of processing incoming satellite imagery as it is collected and generating anomaly alerts within minutes of image acquisition — would significantly increase the operational value of the system. This would require architectural changes to the preprocessing and inference stages to support streaming input and incremental ChromaDB updates.

8.2.5 Active Learning for Anomaly Label Collection

While STAD is designed to operate without labeled anomaly data, incorporating an active learning component that presents the highest-confidence anomaly detections to domain experts for verification could progressively build a labeled dataset that enables the system's anomaly scoring thresholds to be calibrated more precisely over time. This human-in-the-loop enhancement could further reduce the false-positive rate while maintaining or improving recall.

8.2.6 Web-Based Operational Interface

Developing a web-based interface for the STAD system — providing interactive map visualization of anomaly heatmaps, configurable scene selection, temporal playback of object tracks, and one-click anomaly report generation — would make the system accessible to domain analysts and operational decision-makers who do not have the technical background to run the command-line pipeline directly. Such an interface would be a critical step toward transforming STAD from a research prototype into a production-ready operational tool.

REFERENCES

[1] A. Radford, J. W. Kim, C. Hallacy et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proceedings of the 38th International Conference on Machine Learning (ICML), 2021.

[2] Z. Zhang, W. Cheng, D. Xu, X. Wang et al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in Proceedings of the European Conference on Computer Vision (ECCV), 2022.

[3] G.-S. Xia, X. Bai, J. Ding et al., "DOTA: A Large-Scale Dataset for Object Detection in Aerial Images," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018.

[4] Ultralytics, "YOLOv8: A New State-of-the-Art Real-Time Object Detector with Oriented Bounding Box Support," Ultralytics Official Repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[5] Microsoft, "Microsoft Planetary Computer – NAIP 0.6m Collection," Microsoft Azure, 2023. [Online]. Available: https://planetarycomputer.microsoft.com

[6] Chroma AI, "ChromaDB: The Open-Source Embedding Database," ChromaDB Documentation, 2023. [Online]. Available: https://docs.trychroma.com

[7] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," International Journal of Computer Vision, vol. 88, no. 2, pp. 303–338, 2010.

[8] J. Li, C. Wang, H. Zhu et al., "Object-Based Change Detection in High-Resolution Remote Sensing Images Using Multi-Scale Segmentation and Semantic Feature Matching," IEEE Transactions on Geoscience and Remote Sensing, vol. 61, 2023.

[9] Y. Jiang, X. Zhu, X. Wang et al., "R2CNN: Rotational Region CNN for Orientation Robust Scene Text Detection," arXiv preprint arXiv:1706.09579, 2017.

[10] X. Yang, J. Yang, J. Yan et al., "SCRDet: Towards More Robust Detection for Small, Cluttered and Rotated Objects," in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.

[11] D. P. Kingma and J. L. Ba, "Adam: A Method for Stochastic Optimization," in Proceedings of the 3rd International Conference on Learning Representations (ICLR), 2015.

[12] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., "An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale," in Proceedings of the International Conference on Learning Representations (ICLR), 2021.

[13] M. Rudolph, B. Wandt, and B. Rosenhahn, "Fully Convolutional Cross-Scale-Flows for Image-Based Defect Detection," in Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2022.

[14] H. Zhao, H. Li, Z. Zhao, and X. Guo, "Graph Neural Networks for Anomaly Detection in Industrial Internet of Things," IEEE Internet of Things Journal, vol. 8, no. 12, pp. 9767–9779, 2021.

[15] USGS, "NAIP Imagery Program Overview," U.S. Geological Survey, 2022. [Online]. Available: https://www.usgs.gov/centers/eros/science/usgs-eros-archive-aerial-photography-national-agriculture-imagery-program-naip